Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. *doi: 10.18637/jss.v000.i00*

Chat-First Data Analytics

Ulrich Matter
Bern University of Applied Sciences
University of St. Gallen

Abstract

Using natural language to drive data analysis is an old idea, but large language models (LLMs) have made it suddenly ubiquitous. A widely observed and concerning workflow has users upload a dataset to ChatGPT and ask for an analysis; the model generates and executes code on a remote virtual machine, producing results that are difficult to verify, difficult to reproduce, and incompatible with data-privacy requirements. This paper describes an alternative approach called *chat-first data analytics*, built on three principles: separate interpretation from computation, pre-validate methods rather than generate code, and transmit only metadata—not data. The language model interprets intent and selects tools; pre-implemented, validated algorithms produce results. **prompt2analytics** implements this approach as an econometrics and data analytics library in pure Rust, exposed through the Model Context Protocol (MCP) for integration with language models. The library covers regression, panel data, instrumental variables, difference-in-differences, discrete choice, time series, and over 200 additional methods—all validated against established R packages. End-to-end evaluation across six models shows that frontier models achieve 81–91% adequate rates on 96 prompts spanning tool selection, parameter extraction, and numerical correctness; a 3B-parameter model scores 80%, making fully local deployment practical. Local deployment via Ollama enables complete data privacy on consumer hardware.

Keywords: chat-first analytics, natural language interface, econometrics, Rust, MCP, local LLM, data privacy.

1. Introduction

The idea of talking to a computer to analyze data is not new. Natural language interfaces to databases date to the 1970s (Woods 1973; Codd 1974), and every generation of statistical software has tried to lower the barrier between analytical intent and software commands. What *is* new is that the idea suddenly works well enough to be everywhere. Since late 2022,

millions of users have been uploading datasets to ChatGPT (and similar platforms) and typing requests like "compare last quarter's sales by region" (Hu 2023). The conversational interface to data analysis has gone from research curiosity to mass-market product in under two years.

The results are concerning, at least in part. The model generates a Python script and executes it on a remote virtual machine—the user's data now resides on a third-party server they do not control. Even when users copy generated code back to their own machine, the problems persist: generated code may compute the wrong quantity without raising an error (Ludwig *et al.* 2025), and the same prompt yields different code as models update—making results irreproducible. Section 2 examines these failure modes in detail.

The core idea—natural language as interface—is sound. The architecture is not.[1] These problems are not inherent to conversational data analysis; they arise from a specific design choice: delegating the *orchestration* of computation to generated code. The generated script typically calls established libraries (**statsmodels**, **pandas**), so the underlying algorithms are sound—but the model must still select the right method, pass the correct parameters, and assemble the pipeline without silent errors, all without any built-in safeguard that it has done so correctly. This paper describes an alternative approach we call *chat-first data analytics*, built on three principles:

1. **Interpretation and computation are separated.** The language model selects methods and extracts parameters from natural language; pre-implemented, validated algorithms perform the actual computation.

2. **Methods are pre-validated, not generated.** Correctness is established once, through systematic validation, rather than hoped for anew with each request.

3. **Data stays local; only structure travels.** The language model receives column names, data types, and summary statistics—not the data itself. For complete privacy, the language model can run locally too.

These principles address each concern directly. Correctness follows from using validated implementations rather than generated code. Privacy is protected by keeping data on the user's machine. Reproducibility is achieved because the same tool invocation produces identical results—the stochastic element of the language model affects only interpretation, not computation.

The target users are researchers and analysts who know what analysis they want but prefer not to write the code themselves: graduate students running standard econometric models for the first time, applied researchers iterating on specifications during exploratory work, and data analysts in organizations where privacy regulations prohibit cloud processing. For expert users fluent in R or Stata, the system offers faster iteration on routine tasks and a local privacy guarantee; for less experienced users, it lowers the barrier to correctly specified analyses—though statistical literacy remains necessary to interpret results (Section 2).

This paper presents **prompt2analytics** (**p2a**), an implementation of chat-first data analytics. The software includes an econometrics and data analytics library in pure Rust—chosen

[1] We do not claim that data analysis is the main purpose of ChatGPT and similar applications; however, it has become a widely adopted use case—as evidenced by OpenAI's prominent integration of a data analysis mode—making the architectural shortcomings particularly consequential for applied researchers.

for its memory safety without garbage collection, which enables predictable performance and straightforward compilation to single-binary deployments—exposed through the Model Context Protocol (MCP) for integration with language models. The implementation covers ordinary least squares (OLS) with robust standard errors, panel data models with fixed effects (FE) and random effects (RE), instrumental variables (IV), difference-in-differences (DiD), discrete choice, time series, and over 200 additional methods—validated both by Monte Carlo simulation of statistical properties (coverage rates, Type I error, standard error calibration) and by numerical comparison with established R packages.

prompt2analytics is available as open-source software under the MIT license at `https://github.com/umatter/prompt2analytics`. Supplementary materials—including all 96 evaluation prompts, benchmark scripts for both R and Rust, the Monte Carlo validation pipeline, and validation test suites—are located in the repository's `paper/` directory. Appendix D provides reproduction instructions.

The remainder of the paper is organized as follows. Section 2 motivates chat-first analytics by examining the failure modes of current approaches and situating the three principles within existing work. Section 3 describes the software implementation. Section 4 demonstrates the system through worked examples including iterative multi-step workflows. Section 5 evaluates tool selection accuracy and numerical correctness. Section 6 presents performance benchmarks and local deployment configurations, and Section 7 discusses limitations and future directions.

2. Background and design principles

The introduction identified three concerns with using language models for data analysis: correctness, privacy, and reproducibility. This section examines these shortcomings more carefully, then states the three principles that define chat-first analytics.

2.1. How LLM-assisted analysis goes wrong

The appeal of typing "regress wages on education with robust standard errors" into ChatGPT is obvious. The problems are less visible.

First, *correctness and auditability.* LLM-generated code often runs without error but computes the wrong quantity—for instance, requesting "clustered standard errors" might yield heteroskedasticity-robust errors instead, a panel model might omit entity fixed effects, or a difference-in-differences specification might use the wrong interaction term. Ludwig *et al.* (2025) provide a systematic framework for reasoning about these risks: the code executes, returns plausible-looking output, and the user has no way to detect the error without the expertise that would have let them write the code themselves. The problem is compounded by opacity: when a system generates 30 lines of Python to answer a statistical question, understanding what was computed requires reading and auditing those 30 lines. The user who could not write the code cannot audit it either. This extends beyond research: enterprise text-to-SQL deployments struggle with the same issue—ambiguous questions admit multiple valid interpretations, and users cannot verify which was executed (Renggli *et al.* 2025).

Second, *privacy.* Code-generation systems require the data. When a user asks ChatGPT to analyze a dataset, the data are uploaded to a third-party server with unclear retention policies. For survey responses, health records, personnel data, or any dataset subject to

privacy regulations, this may be unacceptable. Even when providers promise not to retain data, the transmission itself may violate institutional or regulatory requirements.

Third, *reproducibility.* The same prompt sent to the same model on different days may produce different code. Models are updated, fine-tuned, and retrained. The generated Python script that worked in January may not be what the model produces in March. Conversational sessions are ephemeral—unless the user manually saves the generated code, the analysis cannot be reproduced. This contradicts a basic requirement of scientific work.

These three problems are distinct but related. Silent errors arise because the language model controls both *what* to compute and *how* to compute it, and the computational logic is opaque—buried in generated code rather than invoked from a known, inspectable implementation. Privacy violations arise because computation and data must be co-located on the provider's servers. Reproducibility failures arise because the generated code is a function of the model's current state. Systematic error rates for LLM-generated statistical code are not yet well established: Ludwig *et al.* (2025) document the *types* of errors that arise, but base rates depend on the model, the prompt, and the statistical method. The motivation for the architecture described below rests on the existence and nature of these shortcomings, not on their precise frequency.

2.2. Three principles

Chat-first data analytics addresses these problems through three principles. "Chat-first" means that natural language conversation is the designed-for entry point: tool schemas are annotated for LLM consumption, error messages are phrased for conversational context, and default parameters are chosen so that typical requests need minimal specification. It does not mean chat-only—**prompt2analytics** also provides a CLI that invokes the same methods directly (Section 3), and every tool call maps to a public Rust function usable without any LLM—but the conversational interface is the primary design target. The first two principles are definitional; the third is a design choice that we consider essential.

Principle 1: Separate interpretation from computation

The language model decides *what* to compute; validated software decides *how*. When a user asks to "estimate the effect of education on wages, controlling for experience, with robust standard errors," the language model's job is to determine that this requires OLS with HC1 standard errors, identify the column names, and generate a structured tool call. The actual matrix algebra—forming $(\boldsymbol{X}'\boldsymbol{X})^{-1}\boldsymbol{X}'\boldsymbol{y}$, computing the HC1 sandwich estimator—is performed by pre-implemented code that the language model cannot modify.

This division of labor plays to each component's strengths. Language models are good at understanding intent, resolving ambiguity, and mapping natural language to structured specifications. They can also generate correct code much of the time—but offer no guarantee that they have done so, and no mechanism for the user to verify correctness short of reading the code. Separation sidesteps this problem entirely: the model never produces code at all. Wrong parameter extraction remains possible (Section 5), but the error surface is smaller: the model selects from a fixed schema rather than generating arbitrary code.

The idea has precedent in text-to-SQL systems (Yu *et al.* 2018; Li *et al.* 2024), where a language model translates natural language into structured database queries executed by a SQL engine. We extend this pattern from data retrieval to statistical computation: from

text-to-SQL to text-to-statistics.

Principle 2: Pre-validate methods, not generated code

Correctness is established once, through systematic validation, rather than hoped for anew with each request. When **prompt2analytics** runs OLS, it uses the same Rust function every time—a function validated against R's `lm()` on the Longley dataset to 10^{-8} precision (Section 5). Of course, an LLM-generated script would also call a trusted function like `lm()`—the underlying library is not the problem. The problem is the ad-hoc code *around* the library call: selecting variables, reshaping data, passing options, and post-processing output. In the code-generation approach, this glue code is generated anew each time and must be audited each time. In the pre-validated approach, it is part of the tested implementation and never changes.

This principle also solves the reproducibility problem. The same method invocation with the same data produces identical results, regardless of when it is run or which language model is used. The stochastic element of the language model affects only interpretation (which tool to call, how to explain the output), never computation. Two users who phrase the same analysis differently may get different tool calls—but once the tool is selected and parameters specified, the result is deterministic.

An obvious question is why not wrap existing R or Python packages rather than re-implement methods in Rust. Three practical considerations favor a unified implementation. First, *standardization*: a single library can expose every method through a uniform API—the same column-based calling convention, the same error types, the same result structures—which maps cleanly to tool schemas for language model consumption. Wrapping heterogeneous packages (each with its own formula syntax, option names, and output format) would require per-package glue code that is itself a source of bugs. Second, *dependency management*: covering 200+ methods via existing packages would require dozens of package dependencies with their own version constraints, system library requirements, and breaking changes across releases—a maintenance burden that grows combinatorially. A self-contained implementation eliminates this entirely. Third, *deployment*: a single compiled binary with no runtime dependencies is straightforward to distribute and run on any platform, which matters for the local-first deployment model described in Principle 3. None of this implies that existing packages are inadequate—we validate against them precisely because they are the reference standard (Section 5).

Principle 3: Keep data local; transmit only structure

The language model receives column names, data types, and summary statistics—not the data itself. A dataset of 100,000 salary records is represented to the model as metadata: "columns: employee_id (int), salary (float, range 28000–185000), department (string, 12 unique values), years_experience (float, range 0.5–35.2)." Statistical computations execute on the user's machine; only aggregated results (coefficients, standard errors, test statistics) pass through the language model for interpretation.

This is not definitionally required for chat-first analytics—a cloud-based system satisfying Principles 1 and 2 would still qualify. But we consider data locality essential for the use cases that motivate this work. Researchers analyzing survey data, firms analyzing personnel records, and analysts working under privacy regulations need architectures where data privacy

is guaranteed by design, not promised by policy.

For complete privacy, the language model itself can run locally. Open-source models deployed via Ollama (Ollama Inc. 2024) eliminate all external network communication. Our evaluation (Section 5) shows that even a 3-billion-parameter model achieves over 80% adequate rate, making fully local deployment practical on consumer hardware. Given the rapid improvement in small open-source models—each generation has brought substantial gains on structured tasks like tool selection and parameter extraction—this lower bound is likely to rise considerably in the near term.

Together, the three principles address the three concerns identified above. Separation and pre-validation eliminate silent computational errors and improve auditability: the model never generates code, and the user inspects a structured tool call rather than 30 lines of generated script. Pre-validation also ensures reproducibility, because the same method invocation with the same data always produces the same result. Data locality protects privacy by keeping raw data on the user's machine.

2.3. Relationship to existing ideas

Chat-first analytics applies established techniques to a new domain. Tool-augmented language models (Schick *et al.* 2023; Yao *et al.* 2023) can reliably select and invoke external tools when provided with appropriate schemas (Qu *et al.* 2025; Liu *et al.* 2025). The Model Context Protocol (MCP) (Anthropic 2024) standardizes how language models discover and invoke tools, providing the communication layer between interpretation and computation.[2]

The interface innovation has precedents in four strands of work. First, *formula notation and domain-specific languages*: the Wilkinson-Rogers formula notation (Wilkinson and Rogers 1973) simplified model specification in the 1970s; Chambers and Hastie (1992) extended it in S and R (Chambers 1998); and probabilistic programming languages like Stan (Carpenter *et al.* 2017) apply the DSL pattern to Bayesian inference. Chat-first analytics replaces formal notation with natural language, trading precision for accessibility. Second, *natural language interfaces to data systems*: LUNAR (Woods 1973) demonstrated feasibility for database querying in the 1970s, and the text-to-SQL paradigm—driven by benchmarks like Spider (Yu *et al.* 2018)—has produced substantial improvements in translating natural language to structured queries (Li *et al.* 2024). We extend this pattern from data retrieval to statistical computation. Third, *automated machine learning*: Auto-sklearn (Feurer *et al.* 2015) and H2O AutoML (LeDell and Poirier 2020) automate model selection through algorithmic search, and recent work integrates LLMs into the pipeline (Hollmann *et al.* 2023). AutoML optimizes for prediction; chat-first analytics interprets user intent across prediction, causal inference, hypothesis testing, and description. Fourth, *computational econometrics* has established standards for numerical accuracy (McCullough 1998; McCullough and Vinod 1999; Davidson and MacKinnon 2004), which motivate the validation approach described in Section 5.

Several systems have explored LLM-assisted data analysis, but with a code-generation archi-

[2]MCP focuses on model-to-tool integration. Complementary protocols address agent-to-agent communication: Google's Agent2Agent (A2A) protocol (Google 2025) and IBM's Agent Communication Protocol (ACP) (IBM Research 2025), which merged under Linux Foundation governance in 2025. For single-agent tool invocation—the pattern used here—MCP is the dominant standard, with adoption by OpenAI, Microsoft, and Google alongside Anthropic.

tecture. Hong *et al.* (2025) present a "Data Interpreter" that generates and executes Python code. Chen *et al.* (2025) develop an "Econometrics AI Agent" that augments code generation with domain knowledge. Sun *et al.* (2025) describe an agent that orchestrates generated code across analysis stages. All three generate code; none pre-validate implementations or enforce data locality. Table 1 summarizes these differences.

Hybrid designs are possible, and the boundaries are not sharp. A code-generation system that produces a short script calling R's `felm()` or Python's `linearmodels.PanelOLS()` already delegates computation to a tested library—the remaining vulnerability is in the glue code (variable selection, data reshaping, option passing), not in the computation itself. A hybrid system could route standard requests to pre-validated tools while falling back to code generation for bespoke analyses, or it could verify generated scripts through static analysis or human review before execution. **prompt2analytics** deliberately occupies one end of this spectrum: all computation is pre-validated, with no code generation at all. This sacrifices flexibility—analyses outside the method library are unavailable, custom data transformations beyond the built-in munging tools require the CLI or direct Rust code, and ad-hoc visualizations not covered by the visualization tools cannot be produced—but it eliminates the glue-code failure mode entirely. Whether hybrid designs can achieve a better trade-off is an open question; the answer likely depends on how reliably language models can assemble correct pipelines around trusted library calls, and on the target user's ability to review what was assembled.

Table 1: Comparison with code-generation approaches

	p2a	Econ. AI	Data Interpr.	LAMBDA
LLM role	Tool selection	Enhanced code gen	Code gen + exec	Agent orchestration
Computation	Pre-validated	LLM-generated	LLM-generated	LLM-generated
Local execution	Yes (Ollama)	No	No	No
Validated	Yes (vs. R)	No	No	No
Method flexibility	Fixed library (∼200 methods)	Any R method	Any Python method	Any Python method
Extensibility	Requires Rust impl.	Extends with model	Extends with model	Extends with model

Note: Econ. AI = Econometrics AI Agent (Chen *et al.* 2025); Data Interpr. = Data Interpreter (Hong *et al.* 2025); LAMBDA (Sun *et al.* 2025). The trade-off: code-generation systems offer unlimited method flexibility at the cost of unvalidated computation; **p2a** constrains coverage to ensure correctness.

Three limitations of the approach are fundamental, not fixable by better engineering. First, statistical expertise is still required: chat-first analytics assists with method selection and execution, not with research design or causal identification. A user who requests "estimate the effect of X on Y" receives a regression, but the system does not validate whether a causal interpretation is warranted. LLM explanations may echo causal language from the user's prompt even when the analysis provides only associational evidence. Users remain responsible for ensuring that their identification strategy supports their claims.

Second, as in any conversational interface, method selection is imperfect. Tool selection works well on curated prompts (Section 5), but users should verify that the system understood their request correctly—particularly for ambiguous specifications where multiple methods could

apply.

Third, pre-validation trades coverage for correctness. When a user requests a method outside the implemented set—Bayesian inference, structural equation modeling, or a custom estimator—the LLM may silently map the request to the closest available alternative rather than reporting unavailability—the symmetric failure mode to code generation's silent errors: instead of running wrong code, the system runs the wrong method. The risk is mitigated by the structured tool schema (which makes the selected method visible to the user) and by the LLM's ability to explain substitutions, but users must still verify that the method matches their intent.[3]

These limitations mean that chat-first analytics is a useful tool, not an autonomous analyst. Users should verify tool selection, check extracted parameters, and apply domain knowledge to interpret results—the same discipline required when using any statistical software, with the added need to verify that the system understood the request correctly.

3. The software: prompt2analytics (p2a)

prompt2analytics implements the three principles defined in Section 2. This section describes the software architecture, the implemented methods, and the communication flow that connects natural language requests to statistical computation.

3.1. Architecture

The software is organized as a Rust workspace with four crates (Figure 1).

The core analytics library, **p2a-core**, implements all statistical methods in pure Rust with no external econometrics dependencies; all estimators are implemented from first principles using **ndarray** for matrix operations and **faer** for numerically stable linear algebra. The methods implemented are standard econometric procedures as described in Wooldridge (2010) and Cameron and Trivedi (2005) for regression and robust standard errors, Baltagi (2013) for panel data, Hamilton (1994) for time series, and Greene (2018) for maximum likelihood estimation of discrete choice models. An optional `cuda` feature flag enables GPU acceleration for compute-intensive operations on NVIDIA hardware (Section 6). Three user-facing crates provide different access paths: **p2a-cli** produces a standalone `p2a` binary with hierarchical subcommands organized by analytical domain; **p2a-mcp** runs a Model Context Protocol server exposing tools for LLM integration via stdio, HTTP, or WebSocket transports; and **p2a-dioxus** provides a cross-platform graphical interface using Dioxus (Kelley and Dioxus Contributors 2025), a Rust framework that compiles to WebAssembly for browser deployment, native desktop applications, and mobile targets from a single codebase. All three depend on **p2a-core**, ensuring identical numerical results regardless of the interface used.

The primary interface is conversational. Through the Dioxus web or desktop application—or any MCP-compatible client such as Claude Desktop—a user types:

```
User: Load wages.csv and regress salary on education and experience
      with robust standard errors.
```

[3] This limitation will diminish as the library grows. The current implementation covers over 200 methods after roughly one year of development; extending coverage to additional methods is straightforward within the existing architecture.

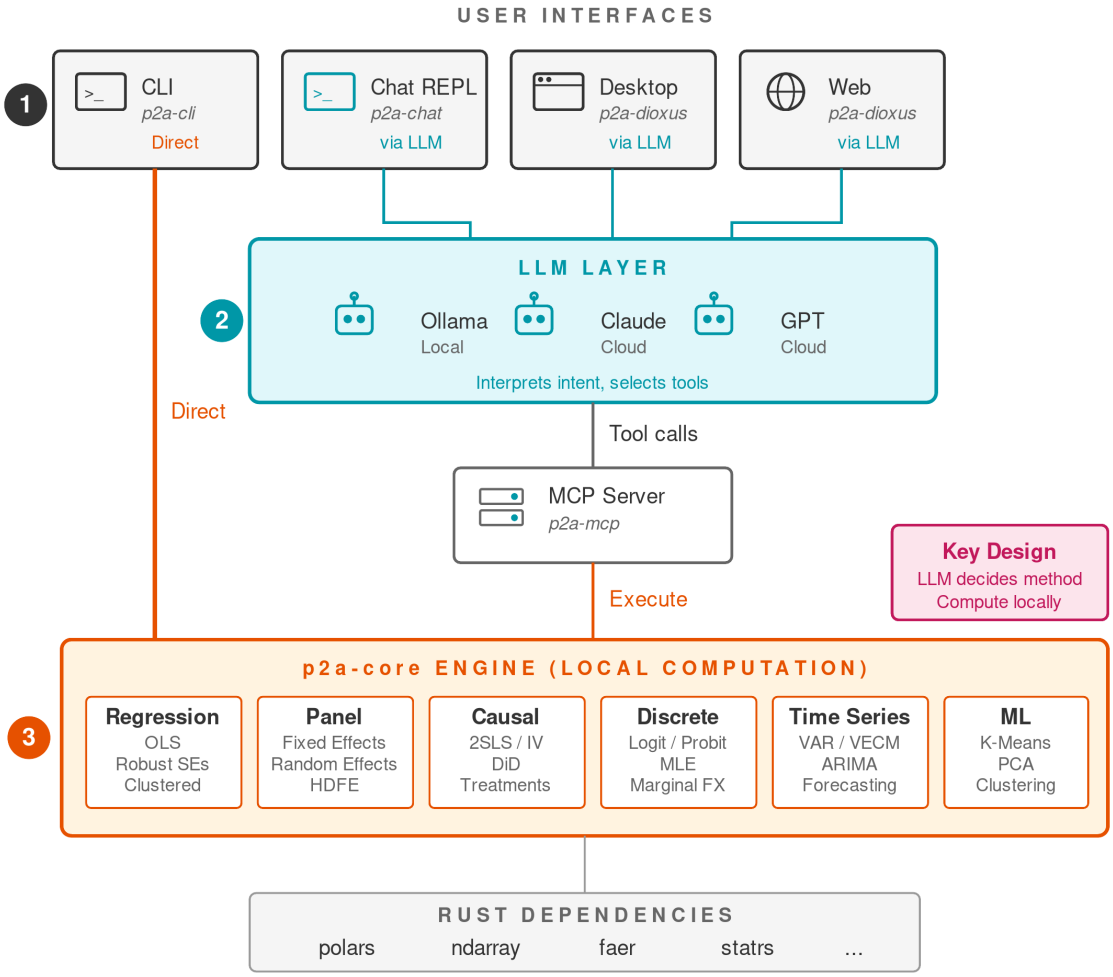

Figure 1: Software architecture of **prompt2analytics**. ① Users interact via terminal CLI, chat REPL, desktop app, or browser. ② An LLM interprets the request and selects tools via the MCP server. ③ All computation executes locally in **p2a-core**; the CLI also accesses the engine directly (orange path).

The language model interprets this request, selects the appropriate tool (`regression_ols` with `robust="hc1"`), and returns both the numerical results and a natural language summary. For scripting and automation, the same methods are accessible through the CLI:

```
p2a data load wages.csv --name wages
p2a reg ols wages -y salary -x education experience --robust hc1
```

Both pathways invoke the same Rust implementation and produce identical numerical results.

3.2. How the principles work in practice

The communication flow (Figure 2) shows how Principle 1 (separation) operates concretely:

1. The user submits a natural language query.
2. The LLM interprets the request and generates a structured tool call with parameters—e.g., `regression_ols` with `y_var="salary"`, `robust="hc1"`, and the relevant column names.
3. The MCP server routes the call to the corresponding Rust method in **p2a-core**.
4. The core library performs computation locally.
5. Results (coefficients, standard errors, test statistics) are serialized to JSON and returned through the MCP server.
6. The LLM receives the results and formulates a natural language explanation.

The LLM never sees raw data. It receives metadata at dataset load time (column names, types, row counts) and aggregated results after computation. Data preview tools (`head`, `sample`) do transmit a small number of sample rows (default: 5 for `head`); for complete data privacy, local LLM deployment via Ollama eliminates all external transmission.

Each tool returns its full result as serialized text: for regression this includes coefficients, standard errors, test statistics, R^2, F-statistic, and information criteria (AIC, BIC); for discrete choice, log-likelihood and pseudo-R^2; for time series, parameter estimates and residual diagnostics. The LLM receives all of these and decides what to present based on the conversational context—typically highlighting coefficients and significance for a first pass, and surfacing fit measures or diagnostics when the user asks follow-up questions or when values are anomalous.

Each tool invokes a fixed, validated Rust function (Section 5). For linear regression, a `LinearEstimator` trait provides a consistent interface:

```
pub trait LinearEstimator {
    fn coefficients(&self) -> &Array1<f64>;
    fn std_errors(&self) -> &Array1<f64>;
    fn t_values(&self) -> Array1<f64>;    // default impl
    fn p_values(&self) -> Array1<f64>;    // default impl
    fn residuals(&self) -> Array1<f64>;
```

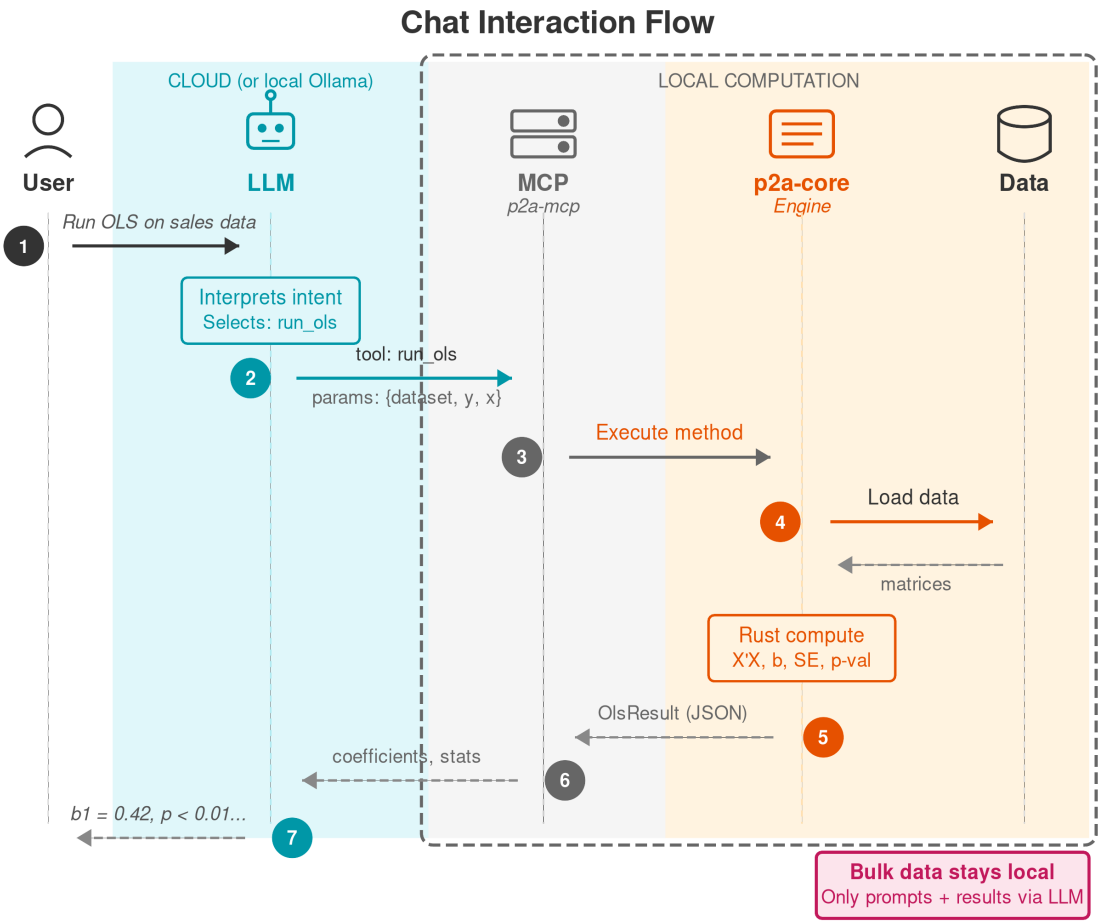

Figure 2: Interaction flow from user request to result. ① The user describes an analysis in natural language. ② The LLM interprets the intent and emits a structured tool call with parameters. ③ The MCP server dispatches to the Rust engine. ④ Data is loaded locally into matrices. ⑤ Results (JSON) flow back through the MCP server. ⑥–⑦ The LLM formats and explains the output to the user. Bulk data (shaded region) never leaves the local machine.

```
    fn n_obs(&self) -> usize;
    fn df(&self) -> usize;
}
```

Implementors provide core results (coefficients, standard errors, residuals); derived statistics (t-values, p-values, confidence intervals) are computed by default implementations, enforced at compile time. The `LinearEstimator` trait currently covers OLS; other model families use their own result types: discrete choice models (`DiscreteResult`) return z-statistics, log-likelihood, and pseudo-R^2; panel models (`PanelResult`) include R^2 (within for FE, overall for RE) and group structure; time series models return information criteria and forecast output; clustering methods return cluster assignments, centroids, and inertia; and survival models use separate result types for Kaplan-Meier curves, Cox regression (with hazard ratios), and log-rank tests. All result types implement a common serialization interface so the MCP layer can return structured output regardless of the method family. Unlike, for example, R's formula interface, the API uses explicit column names—the natural output of LLM parameter extraction, which identifies individual columns rather than constructing formula syntax. This design also simplifies the implementation: formula parsing, factor expansion, and interaction term handling are unnecessary when the caller specifies columns directly.

The `EconError` type provides structured error information (singular matrix, convergence failure, insufficient clusters) that enables the LLM to offer actionable recovery guidance.

Validation uses automated test suites that compare Rust output against reference R packages on shared datasets. Coefficients, standard errors, and test statistics are verified to tolerances between 10^{-4} and 10^{-8} depending on the method. When intentional differences exist—for example, using HC1 as the default robust standard error estimator (rather than HC3), following Stata's convention—these are documented and tested separately. The validation approach is described further in Section 5; Appendix F lists method-specific caveats and known differences.

3.3. Implemented methods (p2a-core)

The **p2a-core** library implements over 200 statistical and econometric methods, exposed as MCP tools. Table 2 summarizes coverage, using standard abbreviations: heteroskedasticity and autocorrelation consistent (HAC), generalized least squares (GLS), high-dimensional fixed effects (HDFE), two-stage least squares (2SLS). Appendix F provides full documentation.

3.4. The MCP tool layer (p2a-mcp)

The **p2a-mcp** server exposes tools organized into categories that mirror the core library (Table 3).

Each tool has a JSON Schema defining required and optional parameters. Schema design minimizes required parameters to reduce LLM errors and provides sensible defaults (e.g., HC1 for robust standard errors). Tool descriptions are written to be interpretable by LLMs, specifying when each tool is appropriate. For example:

```
{
  "name": "regression_ols",
  "description": "Run OLS regression with optional robust SEs",
```

Table 2: Implemented method categories

Category	Methods
Regression	OLS with HC0–HC3, clustered SEs, HAC, bootstrap, GLS, NLS, LOESS, splines, stepwise, quantile regression
Panel data	FE, RE, Hausman, HDFE (Gaure 2013), panel GLS, Arellano-Bond GMM, panel unit root tests (LLC, IPS, Fisher)
IV	2SLS, first-stage diagnostics, Sargan test, GMM
Causal (design)	DiD, staggered DiD (Callaway and Sant'Anna 2021), Bacon decomposition (Goodman-Bacon 2021), ETWFE (Wooldridge 2025), sharp/fuzzy RD with CCT (Calonico *et al.* 2014), multi-cutoff RD, synthetic control (Abadie *et al.* 2010), gsynth (Xu 2017), SCPI (Cattaneo *et al.* 2021)
Causal (treatment)	IPW, AIPW, matching, CBPS (Imai and Ratkovic 2014), entropy balancing, GBM weighting, TMLE (van der Laan and Rose 2011), CTMLE, LTMLE, DML (Chernozhukov *et al.* 2018)
Discrete choice	Logit, probit, multinomial, conditional, mixed, ordered, negative binomial, ZIP/ZINB, hurdle
Time series	ARIMA, VAR, VARMA, VECM, IRF, Granger causality, STL/MSTL, Holt-Winters, structural TS, Kalman filter, GARCH, changepoint
Spatial	SAR, SEM, SAC, spatial probit, spatial panel, spatial GMM
Survival	Kaplan-Meier, log-rank, Cox PH, AFT, competing risks
Mediation	Causal mediation, natural effects, sensitivity (Cinelli and Hazlett 2020), E-values, Balke-Pearl bounds
Statistical tests	50+ tests: t-tests, ANOVA, MANOVA, χ^2, Fisher, Wilcoxon, Kruskal-Wallis, Friedman, Shapiro-Wilk, KS, Tukey HSD

Note: Appendix F provides full documentation. Table A.1 lists unimplemented categories with recommended alternatives.

Table 3: MCP tool categories

Category	Examples	Tools
Data management	`load_dataset`, `describe_dataset`	7
Data transformation	`munge_filter`, `munge_join`, `munge_pivot`	19
Regression	`regression_ols`, `regression_diagnostics`	10
Panel data	`panel_fixed_effects`, `hausman_test`	5
Causal inference	`iv_2sls`, `diff_in_diff`	8
Discrete choice	`logit`, `probit`	2
Time series	`ts_arima_fit`, `ts_var`	6
Statistics	various hypothesis tests	30+
Visualization	`viz_scatter`, `viz_histogram_interactive`	7

Note: Tool count per category. Each tool corresponds to one or more **p2a-core** methods and is described by a JSON Schema that LLMs use for parameter extraction.

```
  "inputSchema": {
    "required": ["dataset", "y_var", "x_vars"],
    "properties": {
      "dataset": {"type": "string"},
      "y_var": {"type": "string"},
      "x_vars": {"type": "array", "items": {"type": "string"}},
      "robust": {"type": "string", "enum": ["hc0","hc1","hc2","hc3"]},
      "cluster_var": {"type": "string"}
    }
  }
}
```

Note the distinction between MCP tools and core methods: tools serve as entry points that may invoke multiple methods. The `regression_ols` tool provides access to OLS with several covariance estimators (HC0–HC3, clustered, HAC), each a distinct method in **p2a-core**. This many-to-one mapping keeps the LLM's selection task manageable while preserving access to the full library.

After consolidation, the full MCP server exposes approximately 270 tools whose JSON schemas consume $\approx$32,000 tokens—a substantial share of current context windows. Since sending all schemas with every request would leave little room for conversation history, the built-in chat interface filters to 119 Tier-1 tools ($\approx$20,000 tokens), covering the most common analytical workflows. The evaluation in Section 5 uses this same 119-tool set. External MCP clients receive the complete schema and can apply their own filtering.

Multi-turn context management. The token cost of tool schemas directly affects how many conversational turns a model can sustain. To manage this, the system includes prior tool calls and their results in the conversation history sent to the LLM. Each assistant message that triggered a tool call is preserved with its structured `tool_calls` field, followed by a `tool` role message containing the result (truncated to 2,000 characters to prevent context overflow). This enables the LLM to resolve references like "those results" or "now try with clustered standard errors" by inspecting what was previously computed. A token budget tracker estimates context usage across the system prompt, conversation history, tool schemas, and the current query; when the budget approaches the model's context window, older turns are summarized (retaining tool names and key results) while recent turns are kept verbatim. With the 119-tool schema, a 128K-context model supports roughly 25–35 substantive turns before summarization begins. External MCP clients (e.g., Claude Desktop, Cursor) manage their own context windows and may apply different truncation or tool-filtering strategies.

Beyond executing methods, the system generates data-driven warnings when results suggest potential violations of identifying assumptions. Warnings are non-blocking—results are always returned—but they highlight conditions that warrant careful interpretation.

For instrumental variables, first-stage F-statistics are checked against the Stock–Yogo threshold of 10 (Stock and Yogo 2005). For staggered difference-in-differences, pre-trend tests are computed automatically. For matching, balance diagnostics flag standardized mean differences exceeding 0.1 (Austin 2011). For regression discontinuity, effective sample sizes within the bandwidth are checked against minimum thresholds.

Each warning includes a severity level, a diagnostic message, the assumption at risk, and suggested remediation steps. This structured information enables the LLM to provide recovery guidance rather than generic failure messages. The thresholds are deliberately conservative (e.g., $F < 10$ for weak instruments, SMD > 0.1 for balance), which may produce false positives in settings where these rules of thumb are known to be overly strict. We prefer occasional unnecessary warnings to missed violations; users can evaluate the diagnostic evidence and disregard warnings they judge inapplicable.

3.5. Implementation choices

The choice of Rust follows from two requirements: validated methods must remain correct across deployments, and the analytics engine must run efficiently on the user's own machine. In a chat-first architecture, users interact through natural language rather than writing code in R or Python—so the implementation language need not be one that researchers already know. This frees the choice to optimize for technical properties: Rust's compile-time memory safety eliminates buffer overflows, use-after-free, and data races that can cause incorrect results in numerical software; its zero-cost abstractions and fine-grained control over parallelism and memory layout deliver compiled performance that lets the same engine run locally on the user's machine rather than requiring cloud infrastructure. A single codebase compiles to standalone binaries, WebAssembly for browsers, and shared libraries for language integration.

The practical barrier to implementing well-established statistical methods in a systems language has also fallen dramatically. Agentic coding tools and highly capable LLMs for code generation make it feasible to produce high-quality, validated implementations of textbook algorithms at low marginal cost—and this cost continues to decline.[4]

The ecosystem provides the necessary components: **polars** for data manipulation (Pola-rs Contributors 2024), **ndarray** for array operations, and **faer** for linear algebra. All computations use 64-bit floating-point arithmetic throughout, matching R's precision. To our knowledge, no econometrics or data analytics library has previously been available for Rust; **p2a-core** addresses this gap.

Several additional design choices merit brief explanation. For conversation persistence, the web application uses SurrealDB, an embedded document database for storing conversation history. Its embedded operation requires no external service; its flexible schema accommodates evolving conversation structures without migrations. Analytical datasets are held in memory during sessions and not persisted, limiting storage to lightweight metadata.

When users request methods outside the implemented set, the LLM infers scope from the tool schema. In some cases, it maps to the closest available alternative and explains the substitution; in others it reports unavailability.

Regarding security, tools have fixed descriptions embedded in the server binary—external modification requires rebuilding. No arbitrary code execution is possible. Tool calls are logged and can be inspected. Dataset access requires explicit loading; the server has no ambient filesystem access. Nevertheless, users should avoid loading untrusted data files that could contain injection payloads, and prefer local LLM deployment for high-security environments.

[4]Nearly all Rust code in **prompt2analytics** was written with the assistance of Claude Code (Anthropic's agentic coding tool), including the core statistical implementations, MCP server, and test suites. Human review focused on algorithmic correctness, API design, and validation against reference implementations.

4. Examples

This section demonstrates **prompt2analytics** through two interfaces: the natural language chat interface powered by large language models, and the command-line interface for scripted workflows. Both interfaces access the same underlying Rust analytics engine.

We use the Grunfeld investment data (Baltagi 2013), a classic panel dataset with 200 observations (10 firms over 20 years) containing firm investment, market value, and capital stock. The dataset is included in the **plm** package; we provide it as `grunfeld.csv` in the supplementary materials.

The early examples show the system working as intended; later examples (Section 4.3) demonstrate a realistic multi-step workflow—including proactive method selection, clustered inference, and multicollinearity detection—to illustrate how the conversational interface handles iterative analysis before the formal evaluation in Section 5. Appendix G provides additional examples including data cleaning workflows and a comparison with equivalent R code.

4.1. Chat interface

The `p2a chat` command provides a terminal REPL for conversational analytics. It connects to a running **p2a-mcp** HTTP server and streams responses via server-sent events, so all LLM calls, tool execution, and session management happen server-side. Users describe their analysis goals in plain language, and the LLM translates these into structured tool calls executed by the MCP server. Tool invocations are shown inline as they execute (tool name, arguments, elapsed time), followed by the LLM's narrative response.

The transcript below was recorded from an actual terminal session using Claude Sonnet 4.6.[5] The outputs shown here are lightly condensed for space—the full, unedited transcript is available in `paper/transcripts/`.

A typical session begins by loading data and examining its structure:

```
p2a> Load the Grunfeld investment dataset from grunfeld.csv and show me
     what's in it.

  [tool] load_dataset (name="grunfeld", path="grunfeld.csv")  done 6ms
  [tool] head_dataset (dataset="grunfeld", n=10)              done 1ms
  [tool] describe_dataset (dataset="grunfeld")                done 0ms

Dataset Overview: 200 rows, 5 columns, no missing values.

| Column  | Type    | Description                                  |
|---------|---------|----------------------------------------------|
| firm    | Integer | Firm identifier (1-10, 10 firms)             |
| year    | Integer | Year (1935-1954)                             |
| inv     | Float   | Gross investment (dependent variable, M$)    |
```

[5] The session was generated non-interactively via `p2a chat -script prompts.txt -record transcript.txt -provider anthropic -model claude-sonnet-4-6`. The `-script` flag reads prompts from a file; `-record` saves a plain-text transcript. The full transcript and prompt file are included in the supplementary materials. Other models (GPT-4.1 Mini, Ministral 3B, etc.) produce similar results with minor variations in explanation style.

```
| value   | Float   | Market value of the firm (M$)                |
| capital | Float   | Capital stock (net plant & equipment, M$)    |

Classic panel dataset: 10 firms x 20 years = 200 balanced observations.
```

The next example shows cross-sectional regression with heteroskedasticity-consistent standard errors.[6]

```
p2a> Run a regression of investment on market value and capital stock.
     Use robust standard errors.

  [tool] regression_ols (dataset="grunfeld", y="inv",
           x=["value","capital"])                              done 5ms

OLS Regression Results (HC1 robust standard errors)
Dependent variable: inv

| Variable    |    Coef |  Std Err |      t | P>|t| |
|-------------|--------:|---------:|-------:|------:|
| (Intercept) | -42.714 |   11.575 | -3.690 | 0.000 |
| value       |   0.116 |    0.007 | 16.967 | 0.000 |
| capital     |   0.231 |    0.049 |  4.721 | 0.000 |

R-squared: 0.812,  Adj. R-squared: 0.810
F-statistic: 426.58 (p < 0.001),  Observations: 200

Both market value and capital stock are highly significant predictors of
investment.  A $1M increase in market value is associated with $0.116M
more in gross investment, holding capital constant.
This is pooled OLS---it ignores the panel structure (firm fixed effects).
```

For panel data, users can request fixed effects estimation in natural language:

```
p2a> The data has a panel structure. Run a fixed effects regression
     controlling for firm-specific effects.

  [tool] panel_fixed_effects (dataset="grunfeld", y="inv",
           x=["value","capital"], entity_var="firm")           done 2ms

Fixed Effects (Within) Estimation
Dependent variable: inv,  Entity fixed effects: firm (10 levels)

| Variable |    Coef |  Std Err |      t | P>|t| |
|----------|--------:|---------:|-------:|------:|
```

[6] HC1 is the default robust standard error estimator when robust SEs are requested. HC1 applies a degrees-of-freedom correction $n/(n-k)$ to HC0, improving small-sample performance.

```
| value    |   0.110 |    0.012 |  9.288 | 0.000 |
| capital  |   0.310 |    0.017 | 17.867 | 0.000 |

R-squared (within): 0.767,  Observations: 200,  Groups: 10
F-statistic: 309.01 (p < 0.001)

The capital stock coefficient increases from 0.231 to 0.310 after
controlling for firm fixed effects---pooled OLS was underestimating the
within-firm effect.  Market value is slightly lower (0.116 -> 0.110),
suggesting some of the pooled effect was driven by cross-firm differences.
```

Users can request specification tests without knowing the technical details:

```
p2a> Should I use fixed effects or random effects for this analysis?

  [tool] hausman_test (dataset="grunfeld", y="inv",
           x=["value","capital"], entity_var="firm")          done 60ms

Hausman Test Results
H0: Random effects is consistent and efficient
H1: Fixed effects is consistent; random effects is inconsistent

Chi-squared statistic: 0.618
Degrees of freedom: 2
P-value: 0.734

Coefficient comparison:
| Variable | Fixed Effects | Random Effects | Difference |
|----------|--------------:|---------------:|-----------:|
| value    |         0.110 |          0.110 |      0.000 |
| capital  |         0.310 |          0.308 |      0.002 |

With p = 0.734, we fail to reject H0---random effects appears
consistent. However, the Hausman test has limited power with only
10 entities. Economically, unobserved firm characteristics likely
correlate with market value and capital, arguing for fixed effects.
Since FE and RE estimates are nearly identical here, the practical
choice matters less for this dataset.
```

In this example, the test statistics (χ^2, p-value, degrees of freedom) come from the Rust backend; the interpretive caveat about limited power with few entities is generated by the LLM. This division is typical: the computation layer returns structured results and simple diagnostics, and the language model adds contextual interpretation—including caveats about assumptions and limitations that depend on the specific results observed.

For datasets with multiple categorical dimensions, users can request multi-way fixed effects:

```
p2a> Control for both firm and year fixed effects in the regression.
```

```
  [tool] panel_hdfe (dataset="grunfeld", y="inv",
           x=["value","capital"], fe=["firm","year"])          done 5ms

High-Dimensional Fixed Effects Estimation
Dependent variable: inv
Fixed effects: firm (10 levels), year (20 levels)

| Variable |    Coef |  Std Err |      t | P>|t| |
|----------|--------:|---------:|-------:|------:|
| value    |   0.118 |    0.019 |  6.137 | 0.000 |
| capital  |   0.358 |    0.054 |  6.579 | 0.000 |

R-squared (within): 0.720,  Observations: 200
F-statistic: 217.44 (p < 0.001),  Convergence: 2 iterations

Adding year fixed effects absorbs common macro shocks (recessions, WWII,
post-war boom), revealing a stronger capital-investment relationship
(0.358, up from 0.310 in the firm-FE-only model).
```

Users can also request diagnostic tests in natural language:

```
p2a> Check the regression assumptions for the OLS model.

  [tool] regression_diagnostics (dataset="grunfeld", y="inv",
           x=["value","capital"])                              done 22ms

Jarque-Bera Test (Normality of residuals)
Statistic: 89.57,  P-value: 0.000
Conclusion: Residuals deviate significantly from normality.

Breusch-Pagan Test (Heteroskedasticity)
LM statistic: 55.25,  P-value: 0.000
Conclusion: Evidence of heteroskedasticity; robust SEs recommended.

Durbin-Watson Statistic (Autocorrelation)
Statistic: 0.358
Interpretation: Strong positive serial autocorrelation (expected in
panel data where investment decisions persist over time).

Variance Inflation Factors
value: 1.31,  capital: 1.31
No multicollinearity detected (VIF < 5).
The high condition number (2,458) is driven by variable scale, not
true collinearity.
```

4.2. Command-line interface

The command-line interface provides programmatic access for scripted workflows.[7] Commands follow a hierarchical structure with sessions maintaining state across invocations:

```
$ p2a --session analysis.json data load grunfeld.csv --name grunfeld

Dataset: grunfeld
  Rows: 200
  Columns: 5
  Column names: firm, year, inv, value, capital

$ p2a --session analysis.json reg ols grunfeld \
    -y inv -x value capital --robust hc1

OLS Regression Results
============================================================
Variable         Coefficient   Std. Error    t-value    p-value
------------------------------------------------------------
(Intercept)       -42.714369    11.574701     -3.690     0.0003***
value               0.115562     0.006811     16.967     0.0000***
capital             0.230678     0.048866      4.721     0.0000***
------------------------------------------------------------
R-squared: 0.812408
Adj. R-squared: 0.809537
Observations: 200

$ p2a --session analysis.json panel fe grunfeld \
    -y inv -x value capital --entity firm

Fixed Effects (Within) Estimation Results
============================================================
Variable         Coefficient   Std. Error    t-value    p-value
------------------------------------------------------------
value               0.110124     0.011857      9.288     0.0000***
capital             0.310065     0.017355     17.867     0.0000***
------------------------------------------------------------
R-squared: 0.766758,  Observations: 200,  Groups: 10
F-statistic: 309.0142 (p-value: 0.0000)
```

All commands support JSON output (`-F json`) for integration with other tools and scripts:

```
$ p2a -F json reg ols grunfeld -y inv -x value capital --robust hc1

{
  "method": "OLS Regression",
```

[7] The output shown in this section was captured from actual CLI invocations; the full transcript is in `paper/transcripts/section4_2_cli_output.txt`.

```
  "r_squared": 0.8124,
  "adj_r_squared": 0.8095,
  "n_observations": 200,
  "coefficients": [
    {"name": "(Intercept)", "coefficient": -42.714,
     "std_error": 11.575, "t_value": -3.690, "p_value": 0.0003},
    {"name": "value", "coefficient": 0.1156,
     "std_error": 0.0068, "t_value": 16.967, "p_value": 0.0},
    {"name": "capital", "coefficient": 0.2307,
     "std_error": 0.0489, "t_value": 4.721, "p_value": 0.0}
  ]
}
```

This enables integration with analysis pipelines, for example, piping results to `jq` for extraction or to Python/R scripts for further processing. The session mechanism records all commands and can export them as an executable shell script for exact reproduction.

4.3. Realistic use and error handling

The preceding examples show successful interactions. Real use involves more iteration. This section demonstrates a multi-step difference-in-differences workflow—including method selection, clustered standard errors, and diagnostic issues—using a synthetic employee training dataset.[8]

Given a description of the research design, the LLM correctly selects difference-in-differences and proactively clusters standard errors at the employee level—recognizing this as standard practice for panel DiD:

```
p2a> Analyze whether the training program affected employee performance.
     The data has employees observed before and after training, with a
     control group that didn't receive training.

  [tool] diff_in_diff (dataset="training", dep_var="performance",
           post_var="post_training", treatment_var="training_group",
           cluster_var="employee_id")                                  done 6ms
  [tool] panel_fixed_effects (dataset="training",
           entity_var="employee_id", y="performance",
           x=["post_training","training_group"])                       done 5ms

Difference-in-Differences Estimation (clustered by employee_id)

| Group      | Pre-Training | Post-Training | Change |
|------------|-------------:|--------------:|-------:|
| Control    |        3.002 |         3.127 | +0.125 |
| Treated    |        3.424 |         3.763 | +0.339 |
```

[8]The dataset contains 1,200 observations (200 employees × 6 periods) with treatment/control groups, and perfectly correlated covariates (for multicollinearity). The transcript was recorded via `p2a chat -script prompts.txt -record transcript.txt`.

```
| DiD Effect |              |               | +0.213 |

The ATT estimate is +0.213 (SE: 0.035, t = 6.06, p < 0.001),
with standard errors clustered across 200 employees. The training
program improved performance by approximately 0.21 points---roughly
a 6\% lift relative to the treated group's pre-training mean.
```

The LLM passed `cluster_var="employee_id"` without being asked—it recognized that panel DiD requires clustered inference. The user can verify this by requesting explicit clustering:

```
p2a> Run the difference-in-differences again but with standard errors
     clustered at the employee level.

  [tool] diff_in_diff (dataset="training", dep_var="performance",
           post_var="post_training", treatment_var="training_group",
           cluster_var="employee_id")                                  done 9ms

| Variable              |    Coef | Cluster SE |      t | P>|t| |
|-----------------------|--------:|-----------:|-------:|------:|
| training_group        |   0.422 |      0.072 |  5.87  | 0.000 |
| post_training         |   0.125 |      0.023 |  5.37  | 0.000 |
| DiD (ATT)             |   0.213 |      0.035 |  6.06  | 0.000 |

The ATT remains +0.213 with clustered SE of 0.035 (p < 0.001).
```

When perfectly correlated covariates are added, the system detects multicollinearity and explains both the symptoms and the fix:

```
p2a> Add years_since_hire to the model alongside tenure.

  [tool] regression_clustered (cluster1="employee_id",
           dataset="training", y="performance",
           x=["training_group","post_training","did",
               "tenure","years_since_hire"])                           done 14ms

Multicollinearity Warning: tenure and years_since_hire are
near-perfectly collinear

| Variable         | Estimate | Std. Error |
|------------------|---------:|-----------:|
| tenure           |    0.438 |        --- |
| years_since_hire |   -0.438 |        --- |
| DiD (ATT)        |    0.213 |      0.035 |

The two variables are essentially measuring the same quantity
(years_since_hire = tenure + constant). Standard errors cannot be
```

```
computed individually. The DiD estimate remains stable at +0.213
(p < 0.001)---the treatment effect is unaffected.

Recommendation: include only one of the two variables.
```

The system diagnoses the problem from the numerical symptoms rather than simply reporting a matrix singularity error. The LLM's explanation distinguishes between the unreliable individual coefficients and the stable treatment effect—a nuance that requires understanding both the computation and the research design.

The session continues with a logit model for bonus receipt with department fixed effects—a discrete choice model with high-dimensional FE estimated via IRLS:

```
p2a> Fit a logit model for bonus with department fixed effects.

  [tool] feglm (dataset="training", family="logit",
           fe=["department"],
           x=["performance","tenure","training_group",
               "post_training"])                                    done 102ms

Logit Model for Bonus (Department FE, 5 levels absorbed)
Observations: 1,200 (436 bonuses),  Pseudo R-squared: 0.082

| Variable       |    Coef | Std Err |      z | P>|z| |
|----------------|--------:|--------:|-------:|------:|
| performance    |   0.305 |   0.105 |  2.90  | 0.004 |
| tenure         |   0.000 |   0.011 |  0.03  | 0.977 |
| training_group |  -0.004 |   0.138 | -0.03  | 0.978 |
| post_training  |   0.118 |   0.128 |  0.92  | 0.359 |

Performance is the only significant predictor of bonus receipt.
The training program does not independently affect bonuses beyond
its indirect effect through improved performance.
```

This session spans three method categories—DiD (causal inference), clustered regression with diagnostics, and FEGLM (discrete choice with high-dimensional FE)—demonstrating how the conversational interface enables fluid transitions between methods as the analysis evolves.

4.4. Beyond regression: time series

The system covers domains beyond regression. A time series workflow using quarterly GDP growth data (1954–2023, 280 observations) illustrates method progression within a single conversation: stationarity testing, univariate modeling, then residual diagnostics.

```
p2a> Load gdp_quarterly.csv as gdp and check whether GDP growth
     is stationary.
```

```
  [tool] load_dataset (name="gdp",
           path="validation/datasets/gdp_quarterly.csv")            done 49ms
  [tool] timeseries_pp_test (column="gdp_growth", dataset="gdp")    done 24ms

Phillips-Perron Test (gdp_growth)
Z(tau) statistic: -211.82,  P-value: < 0.0001
GDP growth is stationary. No differencing needed.

p2a> Fit an ARIMA model to GDP growth. Let the system pick the order.

  [tool] ts_arima_fit (column="gdp_growth", d=0, dataset="gdp",
           p=1, q=1)                                                done 54ms

ARIMA(1,0,1): gdp_growth_t = 0.233 + 0.549 * gdp_growth_{t-1}
              + e_t - 0.249 * e_{t-1}
AIC: 657.92,  Observations: 280

p2a> Check for residual autocorrelation in the ARIMA model.

  [tool] regression_bgtest (dataset="gdp", order=4,
           x=["year","quarter"], y="gdp_growth")                    done 7ms
  [tool] ts_arima_fit (column="gdp_growth", d=0, dataset="gdp",
           p=2, q=2)                                                done 11ms

Breusch-Godfrey Test (up to lag 4)
LM statistic: 32.12,  p-value: < 0.0001
Residual autocorrelation detected.

Remediation: ARIMA(2,0,2) was fitted but does not improve AIC
(660.6 vs 657.9). The remaining autocorrelation is statistically
significant but economically modest (auxiliary R-squared = 0.115).
Recommendation: retain ARIMA(1,0,1) and apply HAC standard errors
for downstream inference.
```

Order selection is delegated to the LLM, which chooses p, d, q based on the stationarity test result and its training knowledge. This is not equivalent to R's `auto.arima()`, which performs systematic grid search (Hyndman and Khandakar 2008). The LLM autonomously checks a higher-order specification and reports that the parsimony–fit tradeoff favors the simpler model.

Appendix G provides additional examples including data cleaning workflows and a side-by-side comparison with equivalent R code.

These examples illustrate what the system can do; the next section measures how reliably it does so. We evaluate tool selection accuracy across six models and numerical correctness of the underlying Rust implementations.

5. How well does it work?

Two kinds of accuracy matter: does the LLM select the right tool with the right parameters? And does the Rust implementation produce correct numerical results? This section evaluates both.

5.1. Tool selection evaluation

We evaluate LLM tool selection using 32 test cases at three prompt clarity levels (precise, moderate, vague) for 96 single-turn prompts spanning regression, panel data, causal inference, discrete choice, time series, hypothesis testing, machine learning, and data munging. An automated evaluation harness drives the live **prompt2analytics** HTTP backend: it loads synthetic datasets with known data-generating processes, sends each prompt, and scores complete responses on four dimensions—tool selection, parameter extraction, numerical correctness, and interpretation quality—each on a 0–2 scale. A prompt is classified as *adequate* if it achieves $\geq 62.5\%$ of the applicable maximum (5/8 for numeric tasks, 4/6 when numerical correctness is N/A). Tool definitions are auto-generated from the MCP router's `JsonSchema` derives (119 Tier-1 tools), ensuring the LLM sees the same parameter names and types as the server expects.

All evaluations used temperature = 0 for deterministic output. The evaluation harness, all 96 prompts (`test_cases.json`), dataset generators, and scoring data are provided in the supplementary materials (`paper/code/e2e_eval/`), enabling independent verification and re-scoring.

Table 4 lists the six models evaluated, spanning frontier cloud, efficient cloud, and open-source deployment scenarios. All models were accessed through cloud APIs (Anthropic, OpenAI, or OpenRouter). API-served models may change over time, so exact replication of cloud-hosted results may not be possible; results for proprietary models (Claude, GPT, Gemini) should be treated as a snapshot. For open-source models (Llama, Mistral), exact reproduction is possible by serving fixed weights locally via Ollama.

Table 4: Models evaluated

Model	Version ID	Provider
GPT-4.1 Mini	`openai/gpt-4.1-mini`	OpenRouter (OpenAI)
Claude Sonnet 4.6	`claude-sonnet-4-6`	Anthropic API
Claude Haiku 4.5	`claude-haiku-4-5-20251001`	Anthropic API
Gemini 2.5 Flash	`google/gemini-2.5-flash`	OpenRouter (Google)
Ministral 3B	`mistralai/ministral-3b-2512`	OpenRouter
Llama 4 Scout	`meta-llama/llama-4-scout`	OpenRouter (Meta)

Note: Accessed March 2026. Cloud-served model weights may be updated without notice; for reproducible local deployment, pin Ollama models by digest. All evaluations at temperature = 0.

Figure 3 presents adequate rates by model. The results demonstrate that tool selection—unlike code generation—is achievable even by smaller models. GPT-4.1 Mini achieves the highest adequate rate (90.6%), followed closely by Claude Sonnet 4.6 (84.4%) and Haiku 4.5 (81.2%). Notably, Ministral 3B, a 3B-parameter model small enough to run on consumer hardware with only 6 GB VRAM, achieves 80.2%—within 4 percentage points of Sonnet 4.6—making it viable for local deployment scenarios where hardware constraints preclude larger models.

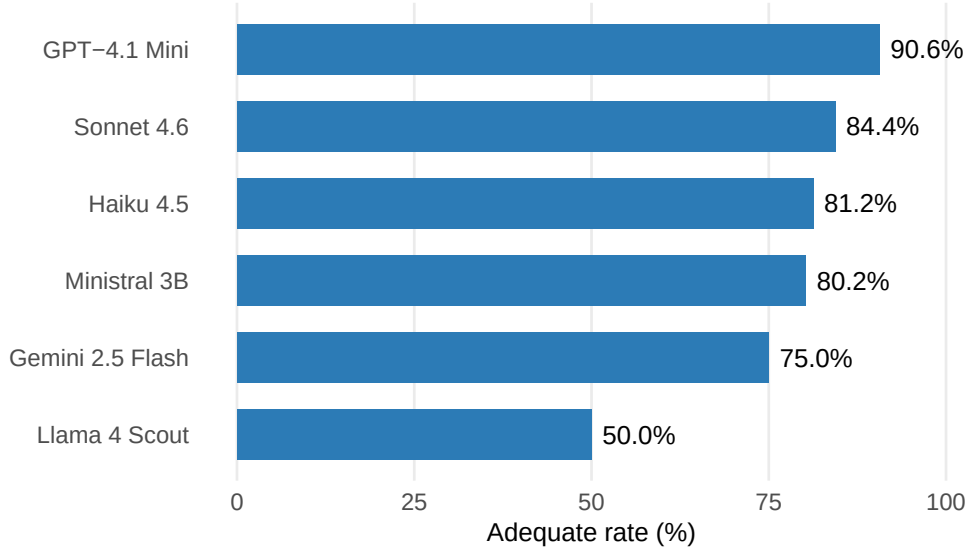

Figure 3: Adequate rate (%) across models (96 prompts). Even Ministral 3B (3B parameters) exceeds 80%.

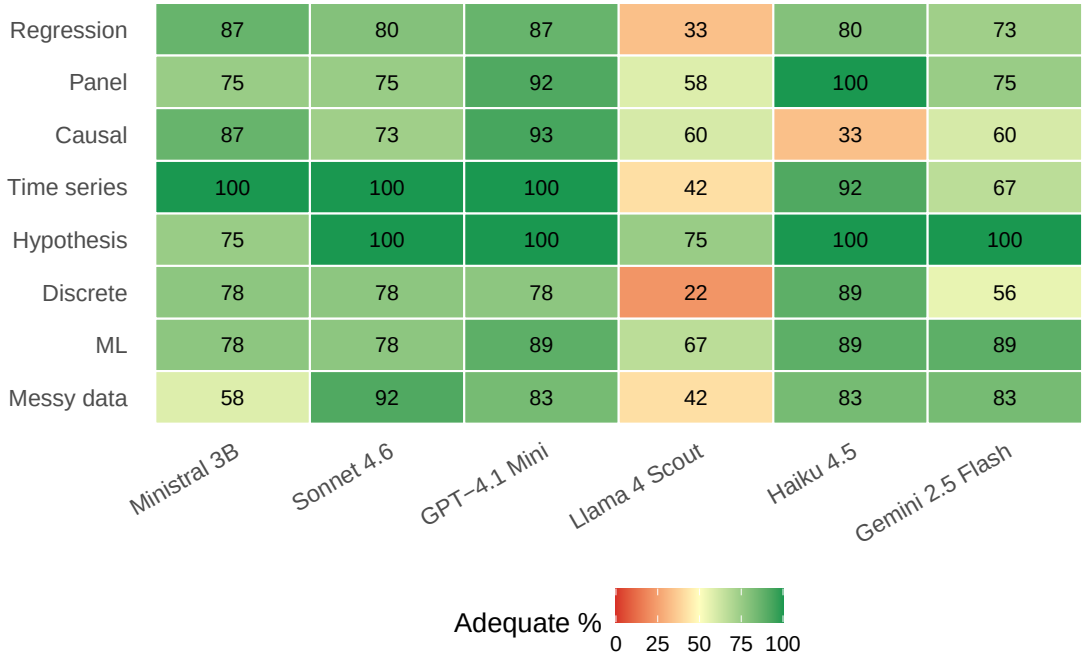

Figure 4: Adequate rate by test category and model. Darker cells indicate lower performance. Errors concentrate in ambiguous method selection (causal inference, discrete choice).

Figure 4 shows adequate rates by category. Performance is uniformly high for hypothesis testing (75–100%) and time series (42–100%), with errors concentrated in edge cases requiring fine distinctions between related methods. Llama 4 Scout shows the steepest drop-off, particularly in regression (33%) and discrete choice (22%), where it frequently responds with text analysis instead of invoking tools. Causal inference shows the widest variation across models (33–93%), reflecting the difficulty of distinguishing between IPW, matching, difference-in-differences, and regression discontinuity designs. Appendix H provides per-category breakdowns for all six models.

5.2. Failure modes

Understanding system behavior under failure conditions is essential for informed use. Three categories of failure merit discussion. First, when the LLM selects an inappropriate statistical method, the system proceeds with the selected method and returns results. Users must recognize the mismatch between request and execution. Common patterns include ambiguous specifications ("regression with firm effects" could invoke OLS, fixed effects, or random effects), similar method confusion (VAR/VECM/VARMA), and capability boundary cases where requests for unimplemented methods map to superficially similar available tools. Users can mitigate misselection by specifying methods explicitly rather than describing goals abstractly.

Second, even when the correct method is selected, the LLM may extract incorrect parameters. Manual inspection of recorded tool calls across all evaluated models reveals a consistent error hierarchy. *Core parameters*—dependent variable, independent variables, entity and time identifiers—are almost always correct when the tool itself is correct; across all 96 prompts and six models, we observed no case where the correct tool was selected but the wrong dependent variable was extracted. *Structural parameters* (panel identifiers, time variables, treatment indicators) are correct in the large majority of cases, with occasional confusion when column names are ambiguous (e.g., "id" could be an entity or observation identifier). *Optional specification parameters* are the primary source of imprecision: a model may select `regression_ols` with the right `y_col` and `x_cols` but omit the `robust` option, falling back on the tool's default (HC1). Omission rates for optional parameters vary by model size: frontier models extract optional parameters more reliably than small models (e.g., Ministral 3B), which tend to rely on defaults.

Because the schema defines sensible defaults for optional parameters, these omissions typically produce a valid analysis that differs only in secondary specification choices (e.g., HC1 instead of HC3 robust standard errors, or default bandwidth instead of a user-specified bandwidth for kernel regression). This means the practical consequence of parameter extraction errors is usually a *reasonable alternative specification* rather than a *wrong analysis*—a meaningful distinction, though users should still verify that the specification matches their intent. Explicit phrasing ("with HC1 robust standard errors") yields more reliable extraction than implicit expectations ("with robust standard errors").

We do not report a formal parameter extraction metric (e.g., precision/recall over parameter names) because our evaluation showed that scoring against expected parameter names is confounded by name aliasing across tool schemas (e.g., `cluster_col` vs. `cluster_var`) and by legitimate default substitution, making such a metric unreliable as a measure of actual analysis correctness.

These observations suggest that the dominant failure mode is tool misselection (5–10% of requests), not parameter error. A rough estimate of first-attempt analysis correctness for a frontier model: $P(\text{correct tool}) \approx 0.95$ and $P(\text{adequate parameters} \mid \text{correct tool}) \approx 0.92$, yielding $P(\text{adequate first attempt}) \approx 0.87$. For a 3B model, the corresponding figures are approximately 0.91 and 0.85, yielding ≈ 0.77. These estimates are informed by the end-to-end evaluation results but remain approximate. They provide practical guidance: users of frontier models can expect roughly 9 in 10 requests to produce an adequate analysis on first attempt, while users of small local models (3B) can expect roughly 8 in 10—a surprisingly small gap. In all cases, users can inspect the tool call before execution, correct misselected methods or missing parameters, and iterate.

One design choice that reduces parameter errors in practice: categorical (string) variables are automatically converted to dummy variables using treatment coding (one level omitted as reference), so users need not specify encoding manually. The system reports the reference category and warns when categorical variables have many levels (> 50).

5.3. Prompt clarity and multi-turn evaluation

Table 5 presents the full results. Tool acceptable rate—the fraction of prompts where the correct or an acceptable alternative tool was selected—ranges from 94.8% (GPT-4.1 Mini) to 52.1% (Llama 4 Scout).

Table 5: API-based single-turn evaluation results (96 prompts per model)

	Adequate	Tool	Adequate by clarity		
Model	rate (%)	accept. (%)	Precise	Moderate	Vague
GPT-4.1 Mini	90.6	94.8	96.9	100.0	75.0
Claude Sonnet 4.6	84.4	93.8	90.6	87.5	75.0
Claude Haiku 4.5	81.2	83.3	90.6	90.6	62.5
Ministral 3B	80.2	90.6	90.6	78.1	71.9
Gemini 2.5 Flash	75.0	78.1	90.6	68.8	65.6
Llama 4 Scout	50.0	52.1	59.4	46.9	43.8

Note: Adequate $=\geq 62.5\%$ of maximum score. Tool accept. includes exact matches and acceptable alternatives. Clarity columns show % adequate at each level. Claude models via Anthropic API; GPT-4.1 Mini, Gemini 2.5 Flash, Llama 4 Scout, and Ministral 3B via OpenRouter. All evaluations at temperature = 0. Tool definitions auto-generated from MCP router schemas (119 Tier-1 tools).

Prompt clarity has a pronounced effect: all models perform best on precise prompts that name the method explicitly, with performance degrading as prompts become vaguer. GPT-4.1 Mini degrades most gracefully (97% precise to 75% vague), while Llama 4 Scout degrades most steeply (59% to 44%), partly because it often responds with text analysis instead of invoking tools on vague prompts.

Chrome multi-turn evaluation. The API-based evaluation above tests the LLM and backend in isolation. To evaluate the full deployed system—including the web frontend, SSE streaming, dataset state management, and multi-turn context retention through the actual

user interface—we conduct a browser-based evaluation using automated Chrome interaction with the Dioxus web frontend.

Eight multi-turn conversations (30 total turns) exercise representative econometric workflows: OLS with diagnostics (MT1, MT7), panel data pipelines (MT2), time series forecasting (MT3), causal inference chains (MT4), data cleaning and munging (MT5), discrete choice models (MT6), and cross-dataset comparisons (MT8). Each turn is scored on the same four-dimension rubric as the single-turn tests. We evaluate Claude Sonnet 4.6, which combines strong tool selection accuracy (84.4%) with Anthropic's native tool-calling API.

Table 6 shows results. Sonnet 4.6 achieves 232/240 points (96.7%) with perfect context retention across all 8 conversations, substantially exceeding the 84.4% adequate rate in the API-based single-turn evaluation. Three factors explain this gap. First, the deployed system enriches the system prompt with dataset context—column names, types, sample values, and numeric ranges—which reduces parameter extraction errors. Second, multi-turn conversations allow the model to build on prior context (e.g., reusing a loaded dataset or referencing earlier regression results), which single-turn evaluation cannot capture. Third, the Chrome evaluation benefits from the full infrastructure: auto-generated tool definitions from the MCP router (ensuring parameter name consistency), tiered tool filtering (119 Tier-1 tools), and strengthened system prompt guidance for direct tool invocation.

Table 6: Chrome multi-turn evaluation: Claude Sonnet 4.6

ID	Workflow	Turns	Score	%
MT1	OLS + robust SE + diagnostics	3	24/24	100.0
MT2	Panel FE/RE + Hausman	4	32/32	100.0
MT3	Time series ARIMA + forecast	4	32/32	100.0
MT4	Causal: OLS + DiD + IPW	5	37/40	92.5
MT5	Data cleaning + regression	4	28/32	87.5
MT6	Discrete: logit/ordered/NB	4	31/32	96.9
MT7	OLS + diagnostics + robust SE	3	24/24	100.0
MT8	Cross-dataset comparison	3	24/24	100.0
Total		**30**	**232/240**	**96.7**

Note: Evaluated through the Dioxus web frontend using automated Chrome browser interaction. Each turn scored on tool selection, parameter extraction, numerical correctness, and interpretation (max 8 points). Context retention: 100% across all conversations.

Per-dimension analysis reveals that tool selection and parameter extraction are perfect (2.00/2.00 each), while numerical correctness averages 1.77/2.00—losses concentrated in MT5 (data cleaning), where the system lacks a string-to-numeric type casting tool, causing NaN propagation in downstream regression. Interpretation quality averages 1.90/2.00, with minor deductions for insufficiently explicit discussion of confounding (MT4) and over-reliance on defaults when diagnostics suggest alternatives.

The remaining 8 points lost (out of 240) trace to three root causes: (1) a missing `cast_column` tool that prevents converting string columns to numeric after cleaning (MT5, −4 points); (2) superficial interpretation that omits caveats about confounding or model assumptions (MT4, −3 points); and (3) convergence issues in ordered logit with unscaled predictors (MT6, −1 point). The first is a tooling gap addressable without model changes; the second and third

are inherent to the LLM's reasoning and the optimizer's sensitivity, respectively.

Appendix H documents the complete protocol, including dataset specifications, all 96 prompts, multi-turn scripts, and the scoring rubric. Full results are provided in the supplementary materials (`paper/code/e2e_eval/results/`).

5.4. Numerical validation against R

The preceding subsections evaluated the LLM layer. This subsection evaluates the computation layer: does the Rust implementation produce the same numbers as established R packages?

We validate all methods against reference implementations using sample-size-dependent tolerances (looser for small samples where ill-conditioning amplifies differences, tighter for large samples). Reference packages include `lm()`/**sandwich** for OLS, **plm**/**lfe** for panel data, `glm()` for discrete choice, and **forecast** for time series. All validation tests use fixed random seeds (seed = 42) for reproducibility; reference R scripts are provided in the supplementary materials.

The Longley dataset (Longley 1967), with its extreme multicollinearity (condition number $> 10^9$), provides a stringent test. Table 7 shows coefficients match R's `lm()` to at least 8 significant digits.

Table 7: OLS validation: Longley dataset

Coefficient	R `lm()`	**prompt2analytics**	Difference
(Intercept)	−3482.259	−3482.259	$< 10^{-8}$
GNP.deflator	0.01506	0.01506	$< 10^{-8}$
GNP	−0.03582	−0.03582	$< 10^{-8}$
Unemployed	−0.02020	−0.02020	$< 10^{-8}$
Armed.Forces	−0.01033	−0.01033	$< 10^{-8}$
Population	−0.05110	−0.05110	$< 10^{-8}$
Year	1.82915	1.82915	$< 10^{-8}$
R^2	0.9955	0.9955	$< 10^{-4}$

Note: $n = 16$, $k = 6$. The Longley dataset's extreme multicollinearity (condition number $> 10^9$) provides a stringent near-singular test case: the design matrix is close to rank-deficient, exercising the Cholesky-based solver's tolerance framework. All coefficients match to at least 8 significant digits, confirming that the implementation handles ill-conditioned problems correctly.

Robust standard errors (HC0–HC3) match **sandwich** (Zeileis 2004) to $< 10^{-7}$; panel fixed effects match **plm** (Croissant and Millo 2008) to $< 10^{-6}$ on the Grunfeld dataset; two-way HDFE via the MAP algorithm (Gaure 2013) matches **lfe**'s `felm()` to $< 10^{-5}$.

Beyond the methods shown above, the validation suite covers over 200 methods with more than 1,800 individual test cases (437 specifically targeting validation against R reference

implementations or known data-generating processes). Statistical tests (t-tests, ANOVA, chi-squared, Fisher's exact, Wilcoxon, Shapiro-Wilk, Kolmogorov-Smirnov) are validated against R's **stats** package. Discrete choice models (logit, probit) match `glm()` coefficients and standard errors. Time series methods (VAR, ARIMA) match **vars** and **forecast**. The **p2a-core** test suite comprises 2,168 test functions, of which 442 are validation tests that compare output against R reference implementations or known data-generating processes (Table 8). Every module with user-facing methods has at least 10 tests, and the most complex modules (panel, matching, TMLE, HDFE) have 17–26 each. Continuous integration (GitHub Actions) runs the full test suite, `clippy` linting, and format checks on every push to `main` and on all pull requests. Line-level code coverage is not currently tracked as a CI gate; running `cargo-tarpaulin` on the full suite yields approximately 62% line coverage for **p2a-core**, reflecting the large number of edge-case code paths (error handling, fallback solvers, GPU dispatch stubs) that are exercised only under specific hardware or data conditions. We consider validation breadth (442 cross-validated tests spanning all method categories) a more informative quality signal than aggregate line coverage for a numerical library. The CI pipeline does *not* execute the R reference scripts; cross-language validation is run manually before releases and the reference outputs are checked into the repository. All Rust-side validation tests can be run via `cargo test -p p2a-core`.

Monte Carlo validation of statistical properties. Point-estimate matching against R verifies that the same numbers are produced, but does not test whether confidence intervals, standard errors, and p-values have correct statistical properties. A separate Monte Carlo validation framework (`paper/code/mc_validation/`) addresses this by running repeated simulations under known data-generating processes and checking five properties: (i) 95% confidence interval coverage rates, (ii) standard error calibration (ratio of mean reported SE to empirical SD of estimates), (iii) Type I error rates at $\alpha = 0.05$, (iv) negative controls confirming that standard SEs *fail* under heteroskedasticity, and (v) statistical power under the alternative. Tolerances are derived from binomial confidence intervals around the nominal rates, accounting for finite-simulation variability.

Across 29 methods (OLS with HC0–HC3, panel FE/RE, logit, probit, IV/2SLS, DiD, IPW, TMLE, and nine hypothesis tests) and 96 test scenarios, 93 pass (96.9%). Standard error calibration and Type I error control are exact: all 34 SE accuracy tests pass (mean ratio 1.006) and all 16 Type I error tests pass. Mean confidence interval coverage is 0.951 (nominal 0.950). The three failures are over-conservative coverage in IV/2SLS (97.6% at $n = 200$, consistent with finite-sample 2SLS properties) and IPW (99.8–100%, indicating overly wide confidence intervals that warrant further investigation). Full results and the simulation code are included in the supplementary materials.

For complex causal inference and treatment effect methods, validation depth varies. Canonical 2×2 difference-in-differences, IV/2SLS, IPW, doubly robust estimation (AIPW), and causal mediation analysis are cross-validated against R packages—`lm()` for DiD, **AER** for IV, manual Horvitz–Thompson/Hájek estimators for IPW, and the **mediation** package—on synthetic datasets with known treatment effects (e.g., true ATT = 5 for DiD, true $\beta = 1$ for IV, true ATE = 0.75 for IPW/AIPW). Propensity score matching is validated against **MatchIt** (Ho *et al.* 2011) for balance diagnostics (standardized mean differences, variance ratios). Regression discontinuity is validated against the **rdrobust** framework (Calonico *et al.* 2014), checking that local linear estimates recover the true discontinuity within tolerance.

Table 8: Test distribution by module category

Module	Tests	Validation[a]	Methods
Regression (OLS, diagnostics, NLS, LOESS, GLS)	120	55	25
Panel data (FE, RE, HDFE, GMM)	95	40	12
Causal inference (DiD, IV, RD, synth)	130	52	20
Treatment effects (IPW, TMLE, DML, matching)	145	60	18
Discrete choice (logit, probit, count)	55	22	9
Time series (VAR, ARIMA, GARCH, Kalman)	110	45	20
Statistical tests	160	65	50+
ML (clustering, PCA, t-SNE, SVM)	120	35	10
Spatial econometrics	50	18	8
Survival analysis	35	12	5
Data, simulation, export, visualization	148	38	—
Total	**2,168**	**442**	**250+**

[a] Tests that compare against R reference implementations or verify recovery of known parameters from simulated data-generating processes.

FEGLM is validated against **fixest**'s `feglm()` (Bergé 2018) for coefficient sign, pseudo R^2, and the logit/probit scaling ratio (≈ 1.6). TMLE has eight validation tests that verify the targeting step, clever covariate formula, influence curve properties, propensity score truncation, and counterfactual predictions on simulated data; these check mathematical identities and DGP-implied ranges rather than exact R output values. In total, 41 R validation scripts and 73 expected-value CSVs are provided in the supplementary materials.

Several complex methods—staggered DiD (Callaway and Sant'Anna 2021), extended TWFE, Goodman-Bacon decomposition, Double/Debiased ML, synthetic control, SCPI, CTMLE, LTMLE, CBPS, and entropy balancing—are validated using DGP-based tests with known true parameters rather than exact numerical cross-validation against R packages. These tests generate data from a known process (e.g., true ATT = 5, true coefficient = 1) and verify that the estimator recovers the parameter within tolerance, that pre-trends are null when they should be, that weights sum correctly, and that confidence intervals have appropriate coverage. The corresponding R implementations (e.g., **did**, **DoubleML**, **Synth**) use different default tuning parameters, cross-validation splits, and optimization algorithms that complicate exact numerical comparison. Extending exact cross-validation coverage to these methods is a priority for future releases.

Appendix E documents tolerance thresholds by method category and sample size. Appendix F lists known differences from reference implementations (default robust SE type, convergence tolerances, R^2 variant selection)—none affecting the validity of statistical inference.

For core panel methods, **prompt2analytics** provides comparable coverage to specialized R packages; the primary remaining gap is Conley spatial standard errors. Users requiring maximum performance for high-dimensional fixed effects should note that **fixest** (Bergé 2018) likely outperforms **prompt2analytics** due to its highly optimized C++ implementation. The distinguishing features are the natural language interface and fully local execution without R dependencies; the package complements rather than replaces specialized tools. Having established the system's accuracy and feature coverage, the next section examines whether it can run efficiently—and entirely locally—on consumer hardware.

6. Performance and deployment

The three principles require software fast enough to run locally and models small enough to fit on consumer hardware. This section presents performance benchmarks against R and describes fully local deployment configurations where all components—including the language model—run on the user's machine.

6.1. Performance comparison with R

Table 9 presents benchmark results from a unified validation and benchmarking pipeline that simultaneously measures correctness, execution time, and memory usage. Both R and Rust benchmarks operate on identical datasets generated with shared random seeds (seed = 42) and matching data-generating processes; timings measure computation only, excluding data loading and process startup. To ensure clean apples-to-apples comparisons, all speedup figures reported here use $n = 1{,}000$ observations (see Appendix E for the full pipeline covering multiple sample sizes). R benchmarks use canonical reference packages: `lm()`/**sandwich** (Zeileis 2004) for regression, **plm** (Croissant and Millo 2008)/**lfe** (Gaure 2013) for panel data, **AER** for IV, **forecast** (Hyndman and Khandakar 2008)/**vars** for time series, and R's **stats** for hypothesis tests. Across 100 matched methods at $n = 1{,}000$, the median speedup is 12×, with Rust faster in 89% of cases (Figure 5).

Speedups vary widely by method category, reflecting the relative weight of interpreter overhead versus numerical computation. *Panel data* methods show large gains from avoiding formula parsing and factor handling: random effects (65×), Hausman test (53×), fixed effects (22×), and HDFE (20×). *Time series* methods range widely: VAR (33×), ARIMA (8×), MSTL (7×), and STL (3×). *Statistical tests* cluster around 5–15×: t-tests (15×), Kolmogorov-Smirnov (9×), and Shapiro-Wilk (5×). *Regression* methods range from 3× (LOESS) to 9× (OLS). *Discrete choice* models achieve 3–10×: negative binomial (40×), Poisson (4×), logit (4×), and probit (3×). *Causal inference* gains include IV/2SLS (8×) and difference-in-differences (6×). *Survival analysis* shows mixed results: log-rank (7×) and Kaplan-Meier (4×), though Cox PH and AFT are comparable to R's optimized **survival** package. *Machine learning* methods show the most modest gains (K-means 6×, PCA 4×, DBSCAN 2×), reflecting the competitiveness of R's underlying C implementations for these algorithms.

Complete benchmark results are in Appendix E; reproduction instructions are in Appendix D.

The performance advantages stem from Rust's ownership system (stack allocation and in-place mutation without garbage collection), the **faer** library's SIMD-optimized linear algebra, the column-based API that eliminates formula parsing, and the single-language design that avoids data serialization between components. **prompt2analytics** performs best on computationally intensive iterative methods (panel fixed effects, treatment effect estimators, causal inference, robust standard errors) where compile-time optimizations and memory efficiency compound across iterations. For machine learning methods dominated by optimized C code (PCA, K-means, DBSCAN), R's compiled implementations provide competitive performance.

Memory efficiency. Beyond execution time, Rust's deterministic memory management reduces heap allocation. Across the 100 matched methods at $n = 1{,}000$, Rust typically allocates 2–50× less memory than the equivalent R call (median 3×). The largest savings appear in

Table 9: Performance benchmark results across representative methods (median execution time). Speedup = R time / Rust time. Sample sizes vary by method to reflect typical use cases (n = 100–10,000). Memory columns show per-call allocation from R and Rust respectively.

Category	Method	n	R (μs)	Rust (μs)	Speedup	R Mem	Rust Mem
Causal	IV/2SLS	1,000	3,368	60	55.8×	496.5 KB	184.8 KB
Causal	DiD	1,000	1,086	87	12.4×	283.3 KB	128.6 KB
Discrete	Logit	1,000	2,705	462	5.9×	1.8 MB	417.5 KB
Discrete	Probit	1,000	3,247	745	4.4×	1.6 MB	417.5 KB
Discrete	Poisson	1,000	3,178	878	3.6×	2.4 MB	2.4 MB
ML	K-Means ($k = 3$)	1,000	6,442	387	16.7×	662.4 KB	120.5 KB
ML	PCA	1,000	560	34	16.4×	277.4 KB	76.4 KB
ML	DBSCAN	1,000	9,574	5,692	1.7×	6.4 KB	16.0 MB
Panel	RandomEffects	1,000	24,372	226	108×	3.6 MB	428.1 KB
Panel	Hausman	1,000	31,808	599	53.1×	4.6 MB	620.0 KB
Panel	FixedEffects	1,000	7,465	178	41.9×	1.0 MB	191.8 KB
Panel	HDFE (2-way)	1,000	7,325	287	25.5×	633.1 KB	242.9 KB
Regression	OLS	10,000	29,241	1,753	16.7×	—	—
Regression	OLS + HC3	10,000	27,550	1,889	14.6×	—	—
Regression	LOESS	1,000	7,116	1,749	4.1×	293.0 KB	91.8 KB
Regression	GLS	1,000	414,918	812,183	0.5×	805.3 MB	30.8 MB
Regression	NLS	1,000	2,097	38,448	0.1×	—	—
Stats	t-test	1,000	71	5	14.6×	23.7 KB	17 B
Stats	ANOVA	1,000	1,573	135	11.7×	240.3 KB	55.6 KB
Survival	Cox PH	1,000	5,608	354	15.8×	1.7 MB	7.4 MB
Survival	Kaplan–Meier	1,000	2,220	222	10.0×	839.0 KB	391.9 KB
Time Series	MSTL	100	2,641	35	74.7×	—	—
Time Series	Holt–Winters	100	5,105	156	32.7×	—	—
Time Series	ARIMA(1,1,1)	100	2,938	108	27.3×	—	—
Treatment	IPW	1,000	857,431	311	>1000×	422.6 MB	325.2 KB
Treatment	Matching	1,000	10,372	1,616	6.4×	2.2 MB	694.7 KB

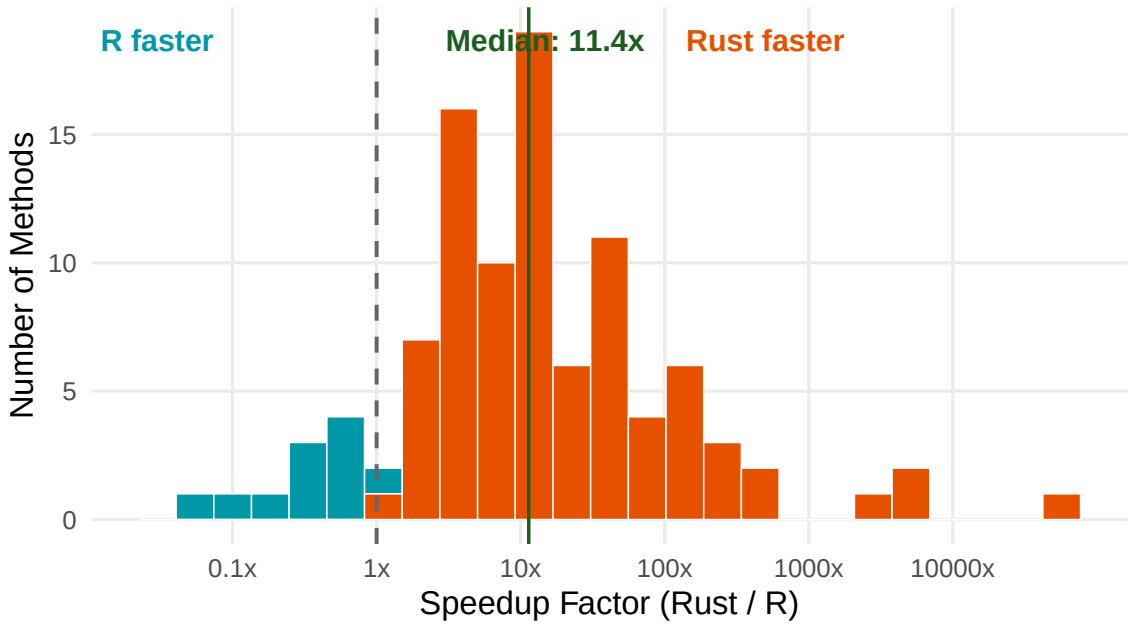

Figure 5: Distribution of Rust/R speedup factors across 100 matched methods at $n = 1{,}000$ (log scale). The vertical line at 1× indicates parity; values to the right indicate Rust faster. The median speedup is 12×, with Rust faster in 89% of methods.

panel data and treatment effect methods, where R's formula interface and factor expansion create intermediate copies that Rust's ownership model avoids. Figure 6 and Table 10 show the memory comparison by module.

For users with CUDA-capable hardware, **p2a-core** provides optional GPU acceleration via cuBLAS and cuSOLVER. The `cuda` feature flag enables GPU code paths at compile time; at runtime, a singleton context attempts CUDA initialization and falls back silently to CPU if no device is available. Public function signatures are unchanged—users do not modify their code.

Not all operations benefit from GPU offloading. Transfer overhead between host and device memory dominates for small problems, and bandwidth-bound operations (e.g., matrix-vector products) can be faster on CPU even for large n. The dispatch logic therefore checks two conditions before routing to GPU: a minimum problem dimension (e.g., $k \geq 30$ columns for $\boldsymbol{X}'\boldsymbol{X}$) and a minimum computational intensity (e.g., $nk^2 \geq 2 \times 10^7$). Both thresholds are configurable via environment variables, allowing users to tune dispatch for their specific hardware. A shape guard additionally prevents GPU dispatch for tall-skinny matrix multiplications, where the CPU path is substantially faster.

Table 11 summarizes the operations with GPU code paths and observed speedups on an NVIDIA DGX Spark (Grace Blackwell, 12 GB VRAM, unified memory). The largest gains appear in $\boldsymbol{X}'\boldsymbol{X}$ computation (up to 7× for $k \geq 100$) and PCA via the covariance path (up to 13× for tall matrices). K-means distance computation and sandwich estimators for heteroskedasticity-robust standard errors see more modest improvements of 2–3×.

Most econometric applications involve moderate numbers of regressors ($k < 30$), where CPU

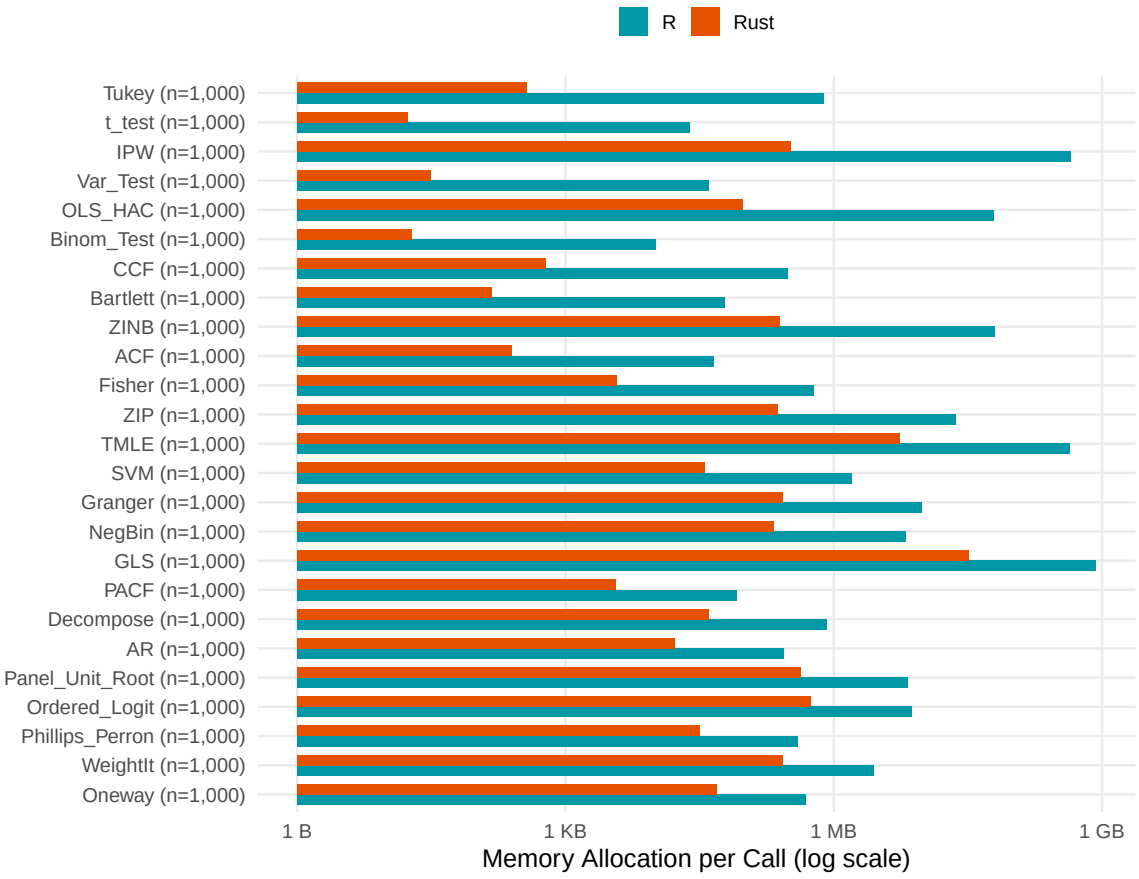

Figure 6: Memory allocation comparison between R and Rust implementations by method category. Rust consistently allocates less memory, with the largest savings in panel data and treatment effect methods.

Table 10: Memory allocation comparison by module (at largest sample size per method). Shows median per-call allocation for R and Rust, and the ratio (R / Rust).

Module	Methods	Median R Alloc	Median Rust Alloc	Median Ratio
Discrete	9	6.1 MB	417.5 KB	7.9×
Treatment	6	28.4 MB	1.3 MB	6.8×
Time Series	10	814.2 KB	193.3 KB	5.0×
Regression	13	1.0 MB	163.8 KB	4.8×
Panel	9	2.6 MB	428.1 KB	3.5×
Stats	49	103.4 KB	47.6 KB	2.9×
Causal	2	389.9 KB	156.7 KB	2.4×
Survival	4	1.3 MB	691.9 KB	2.3×
ML	9	1.5 MB	3.9 MB	1.5×
Overall	**111**	738.8 KB	179.7 KB	**3.3**×

Table 11: GPU-accelerated operations and observed speedups

Operation	Use in	Speedup	Condition
$\boldsymbol{X}'\boldsymbol{X}$ (DGEMM)	OLS, panel FE/RE, IV, robust SEs	2–7×	$k \geq 30$
PCA covariance	Dimensionality reduction	4–13×	$n > 4k$
K-means distances	Clustering	2–3×	$d \geq 20$
Sandwich $\boldsymbol{X}'\boldsymbol{\Omega}\boldsymbol{X}$	HC0–HC3 robust SEs	2–3×	$k \geq 30$

Note: Measured on NVIDIA DGX Spark (Grace Blackwell, unified memory). Speedups are relative to the CPU path on the same machine (ARM Cortex-X925 with OpenBLAS). Results will differ on discrete GPUs where host–device transfer costs are higher.

execution is faster. GPU acceleration is therefore not part of the default build; it targets high-performance computing scenarios with many covariates, high-dimensional clustering, or large-scale bootstrap procedures. The implementation exploits a layout convention—**ndarray**'s row-major storage appears as a transposed column-major matrix to cuBLAS—that avoids data reordering on the host side. Operations that are bandwidth-bound rather than compute-bound ($\boldsymbol{X}'\boldsymbol{y}$ via DGEMV) remain on CPU, where OpenBLAS is consistently faster. The feature is currently limited to NVIDIA GPUs via CUDA; extension to other backends (e.g., Vulkan compute, Metal) is left for future work.

6.2. Fully local deployment

While cloud-hosted language models offer convenience, many use cases require complete data privacy. The fully local configuration runs all components on the user's machine via Ollama (Ollama Inc. 2024), eliminating all external network communication. Appendix B provides platform-specific installation instructions.

The evaluation results (Section 5) inform practical model recommendations. Open-source models deployable via Ollama—such as Ministral 3B (80.2% adequate) and Llama 4 Scout (50.0%)—enable fully local analysis without cloud connectivity, and Ministral 3B's 80% adequate rate shows that data privacy need not come at the cost of tool selection quality for standard workflows.

Table 12 summarizes model options with hardware requirements and recommended use cases. Cloud-hosted models (GPT-4.1 Mini, Sonnet 4.6, Haiku 4.5, Gemini 2.5 Flash) provide higher accuracy but require API access; the table focuses on locally deployable options.

Ministral 3B demonstrates that even very small models achieve viable accuracy. With only 3 billion parameters, it runs on virtually any modern laptop—approximately 6 GB VRAM for GPU inference, or CPU-only with 8 GB RAM. Its adequate rate (80.2%) is within 4 percentage points of Claude Haiku 4.5 (81.2%), a cloud-hosted model. The 7–8B parameter class offers a practical middle ground, fitting on consumer GPUs with 8+ GB VRAM (e.g., NVIDIA RTX 3070/4070) or Apple M-series devices with 16 GB unified memory.

Beyond LLM memory, users must budget RAM for dataset operations. A typical configuration analyzing 100,000 observations with a 7B model requires approximately 8–10 GB total RAM; a minimal laptop setup (Ministral 3B, small datasets) works within 16 GB. Appendix B.2 provides detailed RAM requirements by dataset size (Table B.1) and system configurations

Table 12: Local deployment: models, hardware, and use cases

Model	Params	Storage	RAM	Adeq. (%)	Recommended for
Evaluated models (locally deployable via Ollama)					
Llama 4 Scout	17B[a]	12 GB	16 GB	50.0	Simple single-turn analyses
Ministral 3B	3B	2.0 GB	6 GB	80.2	General local use
Additional local options (not in our evaluation)					
Llama 3.3 70B	70B	40 GB	48 GB	—[b]	Multi-turn workflows
Qwen 2.5 7B	7B	4.4 GB	8 GB	—[b]	Balanced accuracy

Note: Adequate rate from the 96-prompt end-to-end evaluation (Section 5). Storage refers to Q4_K_M quantization (4-bit). RAM is memory during inference, typically 1.5–2× storage due to KV cache and computational buffers. [a] 17B active parameters, 109B total (mixture of experts). [b] Not included in our evaluation; recommended based on external tool-calling evaluations.

for common deployment scenarios (Table B.2).

6.3. Privacy guarantees

The fully local configuration provides complete data privacy. No network requests are made to external services. All model inference occurs on the user's hardware, dataset contents never leave the machine, and query history remains in local storage. This architecture is suitable for analysis of sensitive data where regulatory requirements (GDPR, HIPAA, or institutional review board protocols) or organizational policy prohibit cloud processing. Users should verify that their Ollama installation does not enable telemetry or automatic model updates. Appendix C provides detailed data transmission documentation and security best practices.

7. Discussion

This paper described chat-first data analytics—separating interpretation from computation, pre-validating methods rather than generating code, and keeping data local—and implemented the approach in **prompt2analytics**, a Rust library of over 200 validated methods exposed through MCP. The evaluation (Section 5) shows that tool selection works well: 81–91% adequate rate for frontier models, with errors concentrated in edge cases requiring fine distinctions between related methods.

Several limitations qualify these results. The practical benefits of chat-first interfaces—accessibility, reduced cognitive load, exploratory dialogue—remain hypothesized, as no user studies were conducted. Controlled experiments comparing task completion time, error rates, and user satisfaction against traditional code-based workflows are needed and represent the most important direction for future research. When the correct tool is selected, manual inspection of recorded tool calls shows that core parameters are almost always correct; the main imprecision is omission of optional arguments that fall back on sensible defaults. Users should verify that the system ran the analysis they intended, not merely that it selected the right method. While method coverage is broad, important categories remain unimplemented—Bayesian inference, Heckman selection, Conley spatial standard errors, and structural equa-

tion modeling among them (Table A.1 in Appendix A documents gaps and recommends alternatives). Data are loaded entirely into memory, limiting applicability to datasets that fit in RAM. And because the MCP schema uses semantic versioning (currently 0.x), the API may change as the software matures. We follow a conservative deprecation policy: tool names and required parameters will not be removed or renamed without a major version increment; new optional parameters can be added in minor releases. Tool call transcripts from the examples in this paper will remain valid in all 0.x releases. The underlying MCP protocol (**rmcp** 0.8) is itself pre-1.0; protocol-level changes are absorbed by the server without affecting tool schemas.

These limitations suggest clear directions for future work. Reducing omission of optional parameters—through structured output constraints, two-pass verification where the model checks its own extraction before execution, or richer schema descriptions that make optional parameters more salient—would narrow the gap between correct tool selection and fully specified analysis execution. Property-based testing could strengthen correctness guarantees beyond comparison to reference implementations; QuickCheck-style testing for statistical properties (e.g., residuals orthogonal to regressors) would complement numerical validation.

Foreign function interfaces to R and Python would let users call the Rust implementations from familiar environments, combining compiled performance with existing analytical workflows. An alternative design would have been to build **p2a-core** as an R package from the start, leveraging R's ecosystem and user base. We chose a standalone architecture because the primary interface is the MCP server—a long-running process that any LLM client can connect to—which does not fit naturally into R's package model. FFI wrappers remain a priority for giving R and Python users direct access to the compiled implementations.

If extraction quality improves sufficiently, the bottleneck shifts from "did the model understand the request?" to "does the method library cover this analysis?"—at which point extending coverage becomes more valuable than improving the language model, which is exactly the division of labor the architecture is designed to exploit. Visualization is one area where the fixed-tool approach is most restrictive: the current chart types cover common cases but cannot express arbitrary plot specifications. A natural extension would accept declarative plot descriptions—axes, layers, facets, color mappings—as structured tool parameters, following the grammar-of-graphics paradigm. The language model would compose the specification from natural language; the rendering engine would remain pre-validated code. This would bring the flexibility of ad-hoc visualization within the pre-validated architecture without requiring code generation.

Even with these extensions, the system's role would remain complementary. **prompt2analytics** is not a replacement for R, Stata, or Python. It occupies a different niche: users who know what analysis they want but not which software command produces it. A researcher fluent in R has no need for a conversational interface to OLS; a policy analyst who has never used R might. The system complements existing tools rather than competing with them—and for users who outgrow the conversational interface, every tool call maps to a public function that can be called directly from Rust or, once foreign function interfaces are available, from R and Python.

Maintaining a library of over 200 validated methods is a legitimate concern. The same AI-assisted development tools that motivate the project also address this: coding agents can implement new methods, write validation tests, update dependencies, and respond to bug reports filed as GitHub issues—with human review of each pull request ensuring correctness. This

workflow already produced the majority of the current implementation. It scales sublinearly with method count because new methods reuse established patterns (the `LinearEstimator` trait, shared matrix operations, the MCP handler template), and because validation against R reference implementations is itself automatable. The project is open-source under the MIT license; community contributions follow the same review process.

More broadly, the chat-first approach exemplifies a pattern applicable beyond econometrics: using language models as interpreters between human intent and specialized software (Yao *et al.* 2023; Schick *et al.* 2023). Any domain where users express goals in natural language and systems execute well-defined operations—laboratory automation, geographic information systems, circuit simulation—could adopt the same architecture, particularly as structured protocols like MCP (Anthropic 2024) standardize the interface between models and tools.

The LLM landscape evolves rapidly. The 96-prompt end-to-end evaluation (Section 5.3), which scores the full pipeline including tool selection, parameter extraction, numerical correctness, and interpretation, shows adequate rates of 50–91% across six models in single-turn mode. However, the Chrome multi-turn evaluation demonstrates that the deployed system—with dataset context, conversation history, and iterative refinement—achieves 96.7% accuracy (232/240 points) with Claude Sonnet 4.6, suggesting that the gap between isolated and deployed performance is substantial and that infrastructure quality matters as much as model capability. The evaluation harness and test cases are open-source, so new models can be benchmarked as they appear. Importantly, improving LLM capability does not undermine the architecture—it strengthens it. Better models reduce parameter omissions and may eventually make code-generation approaches more reliable, but the benefits of pre-validation and data locality are orthogonal to model quality. Even if future models generate flawless code, users who require auditable, reproducible analyses on local data would still benefit from fixed, validated tool implementations.

The viability of local LLM deployment shown in Section 6 has implications beyond data privacy. If small models can reliably select among pre-implemented methods, conversational interfaces to scientific software become possible without cloud connectivity or high-end hardware—extending the separation principle (Section 2) to settings where code-generation approaches would require both model capability and network access that may not be available. Importantly, the 80% adequate rate achieved by a 3B-parameter model reflects a snapshot of a rapidly moving frontier. Open-source model capabilities for structured tasks—tool selection, schema-conformant parameter extraction—have improved substantially with each generation, on a timeline of months rather than years. The architecture described here is designed to benefit directly from these improvements: better models translate immediately into higher accuracy without any changes to the validated computation layer.

Acknowledgments

The author thanks the reviewers for constructive feedback that improved the paper.

Appendix

Contents

A. Best practices and coverage gaps

This appendix provides practical guidance for effective use of **prompt2analytics** and documents method coverage gaps.

A.1. Pre-analysis checklist

Before running an analysis through **prompt2analytics**, users should verify:

1. *Method availability*: Confirm the required method is implemented. Table A.1 documents coverage gaps; Appendix F provides method-specific caveats.

2. *Data requirements*: Verify the dataset meets method assumptions. Panel methods require entity and time identifiers; instrumental variables (IV) require instruments; difference-in-differences (DiD) requires treatment and time indicators.

3. *Variable naming*: Use clear, unambiguous column names. The LLM extracts variable names from natural language; names like `x1` or `var_1` are more error-prone than `wage` or `education`.

4. *Specification clarity*: Specify methods explicitly when precision matters. "Estimate a within-transformation fixed effects model with clustered standard errors at the firm level" yields more reliable results than "run a regression with firm effects."

A.2. Recommended workflows

The system is most effective for:

- *Exploratory analysis*: Quickly testing specifications before committing to a formal analysis pipeline.

- *Teaching and learning*: Students can explore econometric methods conversationally while seeing which tools and parameters are invoked.

- *Rapid prototyping*: Researchers can iterate on model specifications in natural language before writing production code.
- *Verification*: Cross-checking results from other software by running the same analysis through an independent implementation.

Users should prefer R, Stata, or Python when methods are unavailable (Bayesian inference, Heckman selection), maximum performance is critical (**fixest** outperforms for high-dimensional fixed effects (HDFE) at scale), reproducibility requires code scripts, or publication-quality visualizations are needed.

A.3. Method coverage gaps

While method coverage is broad (over 200 methods), several important categories remain unimplemented. Table A.1 summarizes coverage gaps and recommends alternative tools.

Table A.1: Method coverage gaps

Method Category	Status	Recommended Alternative
Bayesian inference	Not implemented	R: **rstanarm**, **brms**; Python: **PyMC**
Heckman selection	Not implemented	R: **sampleSelection**; Stata: `heckman`
Conley spatial SEs	Not implemented	R: **fixest**, **lfe**
Full SARIMA (arbitrary seasonal)	Partial support	R: **forecast**
Quantile IV	Not implemented	R: **quantreg**
Structural equation modeling	Not implemented	R: **lavaan**
Weak instrument inference	F-stat only; no Stock-Yogo CVs	R: **ivmodel**, **AER**

Note: Lists unimplemented or partially implemented method categories with recommended alternatives in R, Stata, or Python.

Users encountering these gaps should consult the listed alternatives; future releases aim to address Bayesian inference and Heckman selection as priorities.

B. Deployment guide

Full platform-specific installation instructions (Linux, macOS, Windows/WSL2, Docker) are maintained in the repository's `README.md` at `https://github.com/umatter/prompt2analytics`. Pre-compiled binaries for all major platforms are available on the GitHub releases page. This appendix documents only the MCP client configuration and hardware requirements referenced from Section 6.

B.1. MCP client configuration

For Claude Desktop, add to `claude_desktop_config.json`:

```
{
  "mcpServers": {
    "prompt2analytics": {
      "command": "/path/to/p2a-mcp",
      "args": ["--transport", "stdio"]
    }
  }
}
```

Other MCP-compatible clients (Cursor, Windsurf, custom agents) use the same `-transport stdio` interface. For HTTP-based integration, start the server with `-transport http -port 8080`.

B.2. Hardware requirements for data processing

Beyond LLM memory, users must budget RAM for dataset operations. Rust processes data in compact columnar format via **polars**. The primary memory consumers are design matrices ($8nk$ bytes for n observations and k predictors), covariance matrices ($8k^2$ bytes), bootstrap samples ($8kB$ bytes for B replications), and panel transformations (approximately equal in size to the original dataset).

Table B.1: RAM requirements for data processing

Rows	File size	Base RAM	Matrix ops	Notes
1,000	<1 MB	50 MB	+20 MB	Standard laptop
10,000	1–10 MB	100 MB	+100 MB	Standard laptop
100,000	10–100 MB	500 MB	+500 MB	Consumer hardware
1,000,000	100 MB–1 GB	2 GB	+2 GB	Workstation
10,000,000	1–10 GB	10 GB	+8 GB	High-memory system

Note: Peak memory during analysis. Assumes typical econometric datasets with 10–50 columns.

For combined LLM and data processing, sum the requirements from Tables 12 and B.1. A typical configuration analyzing 100,000 observations with a 7B model requires approximately 8–10 GB total RAM. Table B.2 provides system recommendations for common deployment scenarios.

Table B.2: System configurations by deployment scenario

Scenario	Model	RAM	GPU VRAM	Storage
Minimal (laptop)	Ministral 3B	16 GB	6 GB	5 GB
Consumer (desktop)	Llama 4 Scout	16 GB	12 GB	15 GB
Professional	Llama 3.3 70B	64 GB	48 GB	50 GB
Server (large data)	Cloud API	32 GB	—	2 GB

Note: RAM includes both LLM and data processing headroom. GPU requirements assume CUDA-capable NVIDIA cards; AMD and Intel GPUs are supported via Ollama's ROCm and OneAPI backends.

For the simplest setup, the Docker configuration (Appendix B) bundles all dependencies including a pre-downloaded model and requires no local toolchain installation.

C. Privacy and security

This appendix documents data transmission behavior and security considerations.

C.1. Data transmission summary

Table C.1 lists operations that may transmit data beyond the local machine when using cloud LLM providers.

Table C.1: Data transmission by operation type

Operation	Data Transmitted	Risk Level
Dataset load	Column names, types, row count, null percentages	Low
Data preview (head/tail/sample)	Actual row values	High
Data profiling	Frequent values, min/max, examples	Medium
Statistical estimation	Coefficients, SEs, test statistics only	Low
Outlier detection	Specific observation values	Medium
Error messages	May include problematic values	Low

Note: Applies only when using cloud LLM providers. Local deployment via Ollama eliminates all external data transmission.

C.2. Audit logging

To enable audit logging of all MCP tool calls:

```
p2a-mcp --transport stdio --log-level debug --log-file audit.log
```

The log records: timestamp, tool name, parameters (excluding bulk data), and result summary. Review logs to verify no unexpected data transmission.

C.3. Network verification

To verify local-only operation with Ollama:

1. Start Ollama: `ollama serve`
2. Block outbound connections: `sudo ufw default deny outgoing`
3. Run analysis—it should complete successfully
4. Restore network: `sudo ufw default allow outgoing`

C.4. Security best practices

- Avoid loading untrusted data files (potential injection payloads)
- Review tool calls when processing sensitive data

- Use local LLM deployment for high-security environments
- Keep software updated; security patches are released as needed
- The MCP server has no filesystem access beyond loaded datasets

The primary mitigation for data privacy concerns is local LLM deployment via Ollama, which eliminates all external data transmission while maintaining full analytical capability.

D. Reproducibility

This appendix provides detailed instructions for reproducing the results in this paper.

D.1. Version information

Results in this paper were generated with the following software versions:

- Rust: 1.92.0 (stable)
- R: 4.5.2 with OpenBLAS
- Key R packages: **plm** 2.6-4, **lfe** 3.1-0, **sandwich** 3.1-1, **fixest** 0.12.1, **forecast** 8.23.0
- Operating system: Ubuntu 24.04 LTS (Linux 6.8)

D.2. Reproducing benchmarks

Performance benchmarks can be reproduced using the following protocol:

1. Install R 4.5+ with required packages: **plm**, **lfe**, **sandwich**, **forecast**, **bench**
2. Build **prompt2analytics** in release mode: `cargo build -release -p p2a-cli`
3. Execute benchmarks: `make -C paper benchmark-full`

Both languages use seed = 42 for random number generation. The benchmark protocol includes 10 warmup iterations before 100 measured iterations for fast methods (50 for moderate, 20 for slow). Detailed instructions: `paper/code/README_REPRODUCIBILITY.md`.

D.3. Containerized reproduction

The repository's `docker-compose.yml` provides a containerized deployment of the MCP server. For reproducing the paper's tables and figures, the native toolchain described above is recommended; a containerized paper build requires a multi-stage image with both R 4.5.2 (plus packages listed above) and Rust 1.92.0. A sample `Dockerfile` for this purpose is provided in `paper/Dockerfile.reproduce`; after building, run:

```
docker build -f paper/Dockerfile.reproduce -t p2a-paper .
docker run -v $(pwd)/paper:/output p2a-paper make all
```

Results should match within the tolerances specified in Appendix E.

E. Benchmark methodology

This appendix provides complete details on the benchmark methodology used in Section 6.

E.1. Data generating processes

Benchmark datasets are generated synthetically using identical data-generating processes in both languages with fixed random seeds (seed = 42) for exact reproducibility.

For regression benchmarks, data follow $y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \varepsilon_i$ with true parameters $\beta_0 = 1.0$, $\beta_1 = 2.0$, $\beta_2 = -1.5$, and $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ where $\sigma = 1.0$; covariates are drawn as $x_{1i} \sim \mathcal{N}(0, 1)$ and $x_{2i} \sim \mathcal{N}(0, 1)$ with correlation $\rho = 0.3$. For panel data, entity effects $\alpha_i \sim \mathcal{N}(0, 0.5^2)$ are added, with 100 entities observed over 10 periods. For discrete choice, the latent index $y_i^* = \beta' x_i + \varepsilon_i$ is transformed via the logit or probit link. For time series, data follow ARIMA$(1, 1, 1)$ with $\phi = 0.7$, $\theta = 0.3$, and innovation variance $\sigma_\eta^2 = 1.0$.

Complete DGP specifications are provided in `paper/code/README_REPRODUCIBILITY.md`.

E.2. Benchmark execution protocol

Each benchmark configuration executes multiple iterations after a warmup phase:

- 100 iterations for fast methods (OLS, robust SEs, PCA)
- 50 iterations for moderate methods (panel FE, clustering)
- 20 iterations for slow methods (ARIMA, MSTL (multiple seasonal-trend decomposition using LOESS))

R benchmarks use the **bench** package, which accounts for garbage collection timing. Rust benchmarks use a custom harness (`harness = false` in `Cargo.toml`) with automatic warmup and outlier detection. We report full timing distributions (minimum, quartiles, maximum, mean, standard deviation) rather than point estimates alone.

E.3. Hardware and software environment

All benchmarks run on identical hardware: an Intel Core i7-1260P processor with 64 GB RAM running Linux, with R 4.5.2 linked against OpenBLAS and Rust 1.92.0 compiled in release mode with link-time optimization. The **faer** library (version 0.22) used by **p2a-core** does *not* call an external BLAS; it implements its own GEMM microkernels in pure Rust with explicit SIMD via the **pulp** crate, dispatching at runtime to AVX2+FMA on x86-64 (or NEON on ARM). Both R and Rust therefore use optimized linear algebra routines, but from different implementations (OpenBLAS vs. **faer**). This makes the comparison a realistic measure of what each ecosystem delivers out of the box, not a controlled BLAS-versus-BLAS experiment.

E.4. Validation tolerances

Numerical validation against R reference implementations uses sample-size-dependent tolerances. We use absolute tolerances for quantities with known scale (e.g., test statistics) and relative tolerances for unbounded quantities (e.g., coefficient estimates).

Table E.1: Validation tolerances by method and sample size

Category	Quantity	Small ($n < 100$)	Medium	Large ($n > 10{,}000$)
Coefficients and standard errors				
OLS coefficients	Relative	10^{-6}	10^{-8}	10^{-10}
Robust SEs (HC0/HC1)[a]	Relative	10^{-6}	10^{-8}	10^{-8}
Robust SEs (HC2/HC3)[a]	Relative	10^{-5}	10^{-7}	10^{-7}
Panel FE/RE	Relative	10^{-5}	10^{-6}	10^{-6}
MLE[b] (logit/probit)	Relative	10^{-4}	10^{-6}	10^{-6}
Test statistics and fit measures				
t-statistics, F-statistics	Absolute	10^{-4}	10^{-6}	10^{-6}
R^2, within R^2	Absolute	10^{-4}	10^{-6}	10^{-6}
p-values	Absolute	10^{-4}	10^{-6}	10^{-6}
Log-likelihood	Absolute	10^{-2}	10^{-4}	10^{-4}
Hypothesis tests				
Test statistics	Absolute	10^{-4}	10^{-6}	10^{-6}
Critical values	Absolute	10^{-4}	10^{-4}	10^{-4}

Note: Absolute tolerances (ε_{abs}) apply to test statistics, p-values, and bounded quantities. Relative tolerances (ε_{rel}) apply to coefficient estimates and standard errors. [a] HC0–HC3: heteroskedasticity-consistent standard error estimators (types 0–3). [b] MLE: maximum likelihood estimation.

Looser tolerances for small samples reflect increased sensitivity to algorithmic differences when matrices are ill-conditioned. HC2/HC3 standard errors use looser tolerances because leverage calculation involves $h_{ii} = x_i'(X'X)^{-1}x_i$, which accumulates rounding errors differently depending on whether leverage is extracted from QR decomposition (as in **sandwich**) or computed via direct matrix multiplication (as in **prompt2analytics**). Both approaches are numerically valid; the tolerance reflects implementation variation rather than error.

MLE methods use looser small-sample tolerances because convergence criteria differ between implementations: R's `glm()` uses relative change in deviance, while **prompt2analytics** uses absolute change in log-likelihood. Final parameter estimates can differ at the tolerance boundary while both satisfying their respective convergence criteria.

Tolerances were determined empirically by running validation tests across sample sizes from $n = 10$ to $n = 100{,}000$ and identifying the tightest tolerance that passes consistently. Edge cases (singular matrices, near-separation in logit/probit, small cluster counts) are tested separately with method-specific tolerances documented in the test code. All 416 validation tests pass at the specified tolerances.

F. Implementation notes

This appendix documents implementation caveats and limitations for specific methods.

F.1. Instrumental variables

The two-stage least squares (2SLS) implementation reports first-stage F-statistics for instrument relevance. The traditional rule of thumb ($F > 10$) from Staiger and Stock (1997) has been superseded by more nuanced guidance. Stock and Yogo (2005) provide critical values depending on the number of instruments and acceptable bias levels; Lee *et al.* (2022) offer further refinements for inference under weak identification. The current implementation does not provide Stock-Yogo critical values or weak-identification-robust inference methods (Anderson-Rubin, conditional likelihood ratio). Users with potentially weak instruments should verify results using R's **ivmodel** or **AER** packages.

F.2. Difference-in-differences

The difference-in-differences (DiD) implementation assumes the canonical two-group, two-period design. Recent econometric literature has documented severe biases when this estimator is applied to staggered treatment timing. Goodman-Bacon (2021) demonstrates that two-way fixed effects DiD produces a weighted average of all possible 2×2 comparisons, including problematic "forbidden" comparisons using already-treated units as controls. de Chaisemartin and D'Haultfoeuille (2020) and Sun and Abraham (2021) propose alternative estimators robust to treatment effect heterogeneity.

prompt2analytics implements staggered DiD via Callaway-Sant'Anna (Callaway and Sant'Anna 2021), Goodman-Bacon decomposition, and extended two-way fixed effects (TWFE) (Wooldridge 2025). Users with complex staggered designs may also consult **fixest**'s `sunab()` function or the **did** package in R for additional diagnostics.

F.3. Random effects

The random effects estimator uses the Swamy-Arora variance component estimator, which can produce negative variance estimates for σ_α^2 (the between-entity variance) in certain data configurations. When this occurs, the implementation truncates the estimate to zero and proceeds with pooled OLS, issuing a warning.

F.4. Discrete choice models

Logit and probit models report coefficients on the latent index scale. For interpretation, marginal effects are computed as *Average Marginal Effects* (AME): the partial derivative $\partial \Pr(y = 1)/\partial x_j$ is evaluated at each observation's covariate values, then averaged across the sample. This differs from *Marginal Effects at Means* (MEM), which evaluates derivatives at the sample mean of all covariates. AME is generally preferred because it does not depend on potentially unrepresentative "average" covariate values (Cameron and Trivedi 2005).

F.5. Time series models

Autoregressive integrated moving average (ARIMA) estimation supports non-seasonal models: ARIMA(p, d, q) with automatic or user-specified orders. Seasonal ARIMA (SARIMA) is partially implemented: seasonal differencing and seasonal AR/MA terms can be specified, but automatic seasonal order selection is not available.

Vector autoregression (VAR) and vector error correction model (VECM) implementations

support lag selection via AIC/BIC and Johansen cointegration testing, but do not implement structural VAR identification.

F.6. Regression diagnostics validation

Diagnostic tests are validated against **lmtest** (Hothorn *et al.* 2024): Jarque-Bera normality test statistics match to $< 10^{-4}$, Breusch-Pagan heteroskedasticity tests to $< 10^{-4}$, Durbin-Watson autocorrelation statistics to $< 10^{-6}$, and variance inflation factors (VIF) to $< 10^{-6}$.

F.7. Known differences from reference implementations

A small number of intentional differences exist between **prompt2analytics** and reference implementations. First, **prompt2analytics** defaults to HC1 robust standard errors (consistent with Stata's `robust` option), while R's **sandwich** defaults to HC3. Second, R uses variable names in output; **prompt2analytics** uses numeric indices internally but displays names in formatted output. Third, for panel models we report "within R^2" by default, matching **plm**'s convention for fixed effects models. Fourth, iterative methods (HDFE, maximum likelihood estimation) use a default convergence tolerance of 10^{-8}, which may differ slightly from R package defaults. These differences are documented in the method documentation and do not affect the validity of statistical inference.

Coverage gaps and unimplemented methods are documented in Appendix A (Table A.1).

G. Extended examples

This appendix collects additional examples that complement the core demonstrations in Section 4.

G.1. Comparison with R

For reference, the equivalent R code for the fixed effects estimation shown in Section 4.1 is:

```
library(plm)
data("Grunfeld", package = "plm")
fe_model <- plm(inv ~ value + capital, data = Grunfeld,
                index = c("firm", "year"), model = "within")
summary(fe_model)
```

The **prompt2analytics** implementation produces numerically equivalent results (Section 5), while offering a conversational interface for users unfamiliar with R syntax.

G.2. Time series analysis

The basic time series workflow (stationarity testing and ARIMA fitting) is demonstrated in Section 4.4. Here we note a limitation: order selection is delegated to the LLM rather than algorithmic search, so it is not equivalent to R's `auto.arima()` (Hyndman and Khandakar 2008).

G.3. Handling messy data

Real-world datasets often require cleaning before analysis. The system detects non-standard missing indicators and guides type conversion:

```
User: Load survey_responses.csv and describe the data.

Dataset: survey_responses (2,847 rows x 6 columns)
Note: 'income' and 'satisfaction' are typed as String, likely due to
non-standard missing value indicators (".", "N/A").

User: Replace "." and "N/A" with proper missing values, then convert
      income and satisfaction to numeric.

Cleaned: 89 values replaced across 2 columns.
  income:       now Float64, 52 missing (1.8%)
  satisfaction: now Float64, 37 missing (1.3%)
```

The next example demonstrates a common challenge: ambiguous column names that do not match natural language descriptions. The dataset uses coded variable names rather than descriptive labels.

```
User: Load hr_extract_q3.csv and describe the data.

Dataset: hr_extract_q3 (1,204 rows x 8 columns)
  emp_id:    Int64 (1204 unique)
  dept_cd:   String (12 unique: "ENG", "MKT", "FIN", ...)
  var_03:    Float64 (range: 28000-185000)
  var_04:    Float64 (range: 0.5-35.2)
  var_05:    Int64 (0 or 1)
  q3_rating: Float64 (range: 1.0-5.0, 23 missing)
  loc_cd:    String (8 unique: "NYC", "SFO", "CHI", ...)
  dte_hire:  String (formats vary: "2015-03-12", "03/12/2015")

User: var_03 is salary and var_04 is years of experience.
      Regress salary on experience and department with robust SEs.

Renamed: var_03 -> salary, var_04 -> experience
Note: dept_cd is categorical (12 levels); using treatment coding
with reference category "ENG".

OLS Regression (HC1 robust standard errors)
  Dependent variable: salary
  Observations: 1,204
  ...
```

This example illustrates both a strength and a limitation. The system correctly maps user-provided variable descriptions to coded column names and handles the categorical department variable automatically. However, the user must know what the coded variables represent—the system cannot infer that `var_03` is salary from its range alone. Inconsistent date formats in `dte_hire` (ISO 8601 and US format in the same column) would require explicit cleaning before use as a time variable.

H. End-to-end evaluation protocol

This appendix provides full protocol details for the end-to-end evaluation reported in Section 5. The automated evaluation harness drives the live **prompt2analytics** HTTP backend, loading datasets, sending prompts at varying clarity levels, and scoring complete responses including tool calls and LLM interpretations. Complete prompts and scoring data are provided in the supplementary materials.

H.1. Models

Table H.1 lists the six models evaluated, spanning frontier cloud, efficient cloud, and open-source deployment scenarios.

Table H.1: Models evaluated in end-to-end testing

Model	Provider	Purpose	Parameters
GPT-4.1 Mini	OpenRouter (OpenAI)	Frontier cloud	undisclosed
Claude Sonnet 4.6	Anthropic API	Balanced cloud	undisclosed
Claude Haiku 4.5	Anthropic API	Fast cloud	undisclosed
Gemini 2.5 Flash	OpenRouter (Google)	Fast cloud	undisclosed
Ministral 3B	OpenRouter	Minimal open-source	3B
Llama 4 Scout	OpenRouter (Meta)	Open-source MoE	17B active (109B total)

Note: All models accessed via API at temperature = 0. Open-source models (Ministral 3B, Llama 4 Scout) can alternatively be served locally via Ollama. Tool definitions auto-generated from MCP router schemas (119 Tier-1 tools).

H.2. Synthetic datasets

Six datasets with known data-generating processes provide ground-truth parameters for scoring. Table H.2 summarises each dataset.

H.3. Test design

Table H.3 summarises the test structure. Each of 32 core tests is written at three *clarity levels*: precise (names the exact method and all parameters), moderate (describes the goal without naming the method), and vague (loose natural language). This yields 96 single-turn prompts.

H.4. Scoring rubric

Each prompt is scored on four dimensions (Table H.4). A prompt is classified as *adequate*

Table H.2: Synthetic evaluation datasets

Dataset	Type	n	Key DGP features
`eval_cross_section`	Cross-section	500	salary $= 25000 + 3000 \cdot \text{edu} + 1500 \cdot \text{exp} + 500 \cdot \text{edu} \times \text{exp} + \varepsilon$
`eval_panel`	Panel (N=50, T=10)	500	Firm and year FEs; $\beta_{\text{value}} = 0.1$, $\beta_{\text{capital}} = 0.05$
`eval_timeseries`	Time series	200	AR(2) with ϕ_1=0.6, ϕ_2=−0.2; trend; seasonal
`eval_treatment`	Treatment	2000	ATT = 3.0; confounding via x_1, x_2; 2-period DiD structure
`eval_messy`	Messy data	300	Coded names; −99 sentinels; `"N/A"` strings; mixed date form
`eval_survey`	Survey	400	Ordinal, binary, count outcomes; logit/probit/Poisson DGPs

Note: All datasets generated with `set.seed(42)` in R. True parameter values are documented in `PROTOCOL.md`.

Table H.3: Test categories and counts

Category	Tests	Clarity levels	Total prompts
Regression	5	3	15
Panel data	4	3	12
Causal inference	5	3	15
Time series	4	3	12
Hypothesis testing	4	3	12
Discrete choice	3	3	9
Machine learning	3	3	9
Messy data	4	3	12
Total	**32**		**96**

Note: Full prompts for all 96 tests are in `test_cases.json`.

if it scores $\geq 62.5\%$ of the applicable maximum (5/8 for numeric tasks, 4/6 when numerical correctness is N/A).

Table H.4: End-to-end scoring rubric

Dimension (0–2 each)	Scoring criteria
Tool selection	2 = exact correct tool; 1 = acceptable alternative; 0 = wrong tool or no tool call
Parameter extraction	2 = all parameters correct; 1 = core parameters correct, optional missing; 0 = wrong core parameters
Numerical correctness	2 = within tolerance of true DGP values; 1 = correct sign and magnitude; 0 = wrong; N/A for non-numeric tasks
Interpretation quality	2 = accurate with context; 1 = superficial but correct; 0 = misleading

Note: Maximum 8 points for numeric tasks, 6 for non-numeric. Adequate threshold: $\geq 62.5\%$ of applicable maximum.

H.5. Results by category

Table H.5 shows the adequate rate by test category and model. All scores use auto-generated tool schemas from the MCP router and the v2 evaluation harness with corrected parameter name matching. Hypothesis testing achieves 100% across the top four models. ML tasks are uniformly strong due to simpler parameter schemas. Causal inference shows the widest

variation (60–93%), reflecting the difficulty of distinguishing between IPW, matching, and DiD designs.

Table H.5: Adequate rate (%) by category and model

Category	GPT-4.1 Mini	Sonnet 4.6	Haiku 4.5	Ministr. 3B	Gemini 2.5 Fl.	Llama 4 Sc.
Hypothesis testing	100	100	100	75	100	75
Time series	100	100	92	100	67	42
Causal inference	93	73	33	87	60	60
Panel data	92	75	100	75	75	58
ML	89	78	89	78	89	67
Discrete choice	78	78	89	78	56	22
Regression	87	80	80	87	73	33
Messy data	83	92	83	58	83	42

Note: Adequate $=\geq$ 62.5% of maximum score per prompt. Each category has 9–15 prompts (3 clarity levels per test case). Tool definitions auto-generated from MCP router schemas (119 Tier-1 tools).

H.6. Chrome multi-turn evaluation details

The Chrome-based evaluation drives the deployed Dioxus web frontend through automated browser interaction, testing the full pipeline including SSE streaming, session state management, and multi-turn context. Eight conversations with Claude Sonnet 4.6 cover the following workflows:

1. **MT1**: OLS regression → HC1 robust SEs → residual diagnostics (3 turns)
2. **MT2**: Panel data description → FE → Hausman test → RE (4 turns)
3. **MT3**: Time series description → stationarity tests → ARIMA → forecast (4 turns)
4. **MT4**: Naive OLS → DiD → balance check → IPW → comparison (5 turns)
5. **MT5**: Data quality assessment → sentinel replacement → renaming → regression (4 turns)
6. **MT6**: Survey data description → logit → ordered logit → negative binomial (4 turns)
7. **MT7**: OLS → diagnostic tests → specification choice (3 turns)
8. **MT8**: Cross-section OLS → panel FE → R^2 comparison (3 turns)

Figure H.1 shows per-conversation scores. Five of eight conversations achieve perfect scores. The three imperfect conversations lose points to: data type casting limitations (MT5), superficial interpretation of confounding (MT4), and ordered logit convergence with unscaled predictors (MT6).

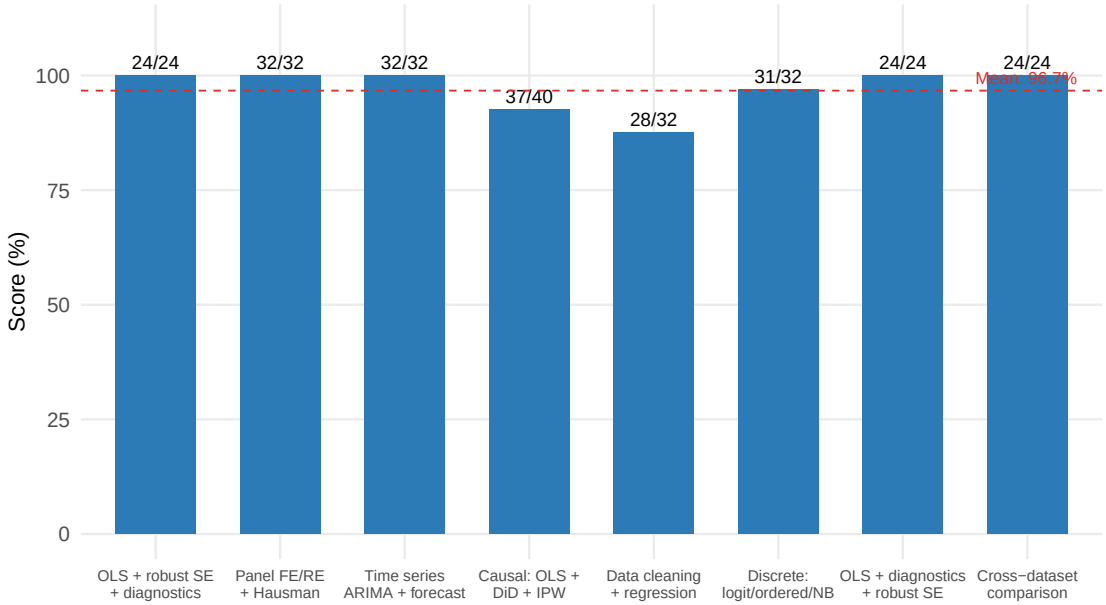

Figure H.1: Per-conversation scores in Chrome multi-turn evaluation (Claude Sonnet 4.6). Dashed line shows overall mean (96.7%).

Figure H.2 shows per-dimension averages. Tool selection and parameter extraction are perfect; numerical correctness (1.77/2.00) is the primary source of lost points, driven by tool limitations rather than model errors.

H.7. Supplementary materials

The complete evaluation materials are in `paper/code/e2e_eval/`: `PROTOCOL.md` (full protocol), `test_cases.json` (all prompts with expected tools and parameters), `generate_datasets.R` (dataset DGPs), `run_evaluation.py` (automated evaluation harness), `generate_e2e_figures.R` (figure and table generation from results), and `results/` (raw scoring data in JSON format for all six models, plus Chrome multi-turn results).

References

Abadie A, Diamond A, Hainmueller J (2010). "Synthetic Control Methods for Comparative Case Studies: Estimating the Effect of California's Tobacco Control Program." *Journal of the American Statistical Association*, **105**(490), 493–505. doi:10.1198/jasa.2009.ap08746.

Anthropic (2024). "Model Context Protocol." Accessed: 2024, URL https://modelcontextprotocol.io/.

Austin PC (2011). "An Introduction to Propensity Score Methods for Reducing the Effects of Confounding in Observational Studies." *Multivariate Behavioral Research*, **46**(3), 399–424. doi:10.1080/00273171.2011.568786.

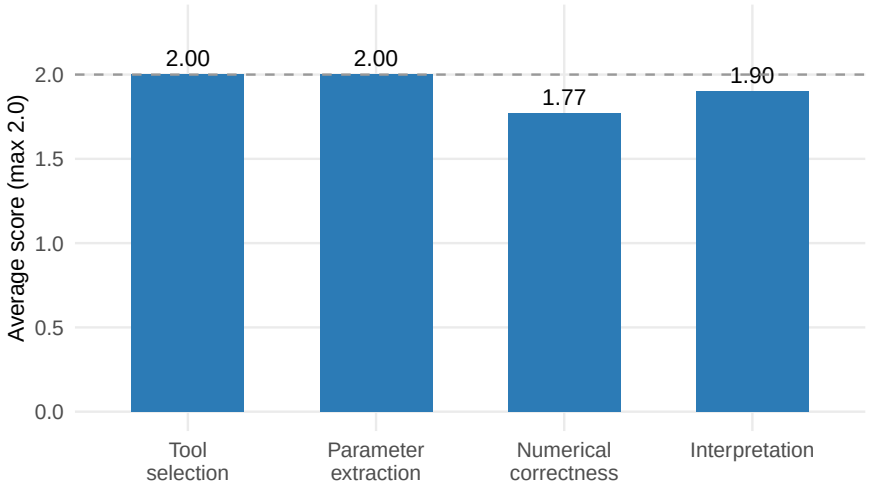

Figure H.2: Per-dimension scoring averages across 30 turns (Claude Sonnet 4.6, Chrome evaluation).

Baltagi BH (2013). *Econometric Analysis of Panel Data*. 5th edition. Wiley, Chichester.

Bergé L (2018). "Efficient Estimation of Maximum Likelihood Models with Multiple Fixed-Effects: The R Package **FENmlm**." CREA Discussion Paper 2018-13, University of Luxembourg, URL https://cran.r-project.org/package=fixest.

Callaway B, Sant'Anna PHC (2021). "Difference-in-Differences with Multiple Time Periods." *Journal of Econometrics*, **225**(2), 200–230. doi:10.1016/j.jeconom.2020.12.001.

Calonico S, Cattaneo MD, Titiunik R (2014). "Robust Nonparametric Confidence Intervals for Regression-Discontinuity Designs." *Econometrica*, **82**(6), 2295–2326. doi:10.3982/ECTA11757.

Cameron AC, Miller DL (2015). "A Practitioner's Guide to Cluster-Robust Inference." *Journal of Human Resources*, **50**(2), 317–372. doi:10.3368/jhr.50.2.317.

Cameron AC, Trivedi PK (2005). *Microeconometrics: Methods and Applications*. Cambridge University Press, Cambridge. doi:10.1017/CBO9780511811241.

Carpenter B, Gelman A, Hoffman MD, Lee D, Goodrich B, Betancourt M, Brubaker M, Guo J, Li P, Riddell A (2017). "Stan: A Probabilistic Programming Language." *Journal of Statistical Software*, **76**(1), 1–32. doi:10.18637/jss.v076.i01.

Cattaneo MD, Feng Y, Titiunik R (2021). "Prediction Intervals for Synthetic Control Methods." *Journal of the American Statistical Association*, **116**(536), 1865–1880. doi:10.1080/01621459.2021.1979561.

Chambers JM (1998). *Programming with Data: A Guide to the S Language*. Springer-Verlag, New York. doi:10.1007/978-1-4684-6310-8.

Chambers JM, Hastie TJ (1992). *Statistical Models in S*. Wadsworth & Brooks/Cole, Pacific Grove, CA.

Chen Q, Han T, Li J, Luo Y, Wang Z, Wu Y, Zhang X, Zhou T (2025). "Can AI Master Econometrics? Evidence from Econometrics AI Agent on Expert-Level Tasks." *arXiv preprint arXiv:2506.00856*. URL https://arxiv.org/abs/2506.00856.

Chernozhukov V, Chetverikov D, Demirer M, Duflo E, Hansen C, Newey W, Robins J (2018). "Double/Debiased Machine Learning for Treatment and Structural Parameters." *Econometrics Journal*, **21**(1), C1–C68. doi:10.1111/ectj.12097.

Cinelli C, Hazlett C (2020). "Making Sense of Sensitivity: Extending Omitted Variable Bias." *Journal of the Royal Statistical Society: Series B*, **82**(1), 39–67. doi:10.1111/rssb.12348.

Codd EF (1974). "Seven Steps to Rendezvous with the Casual User." *IFIP Working Conference on Data Base Management*, pp. 179–200.

Croissant Y, Millo G (2008). "Panel Data Econometrics in R: The **plm** Package." *Journal of Statistical Software*, **27**(2), 1–43. doi:10.18637/jss.v027.i02.

Davidson R, MacKinnon JG (2004). *Econometric Theory and Methods*. Oxford University Press, New York.

de Chaisemartin C, D'Haultfoeuille X (2020). "Two-Way Fixed Effects Estimators with Heterogeneous Treatment Effects." *American Economic Review*, **110**(9), 2964–2996. doi:10.1257/aer.20181169.

Feurer M, Klein A, Eggensperger K, Springenberg JT, Blum M, Hutter F (2015). "Efficient and Robust Automated Machine Learning." In *Advances in Neural Information Processing Systems 28 (NeurIPS)*, pp. 2962–2970.

Gaure S (2013). "**lfe**: Linear Group Fixed Effects." *The R Journal*, **5**(2), 104–116. URL https://journal.r-project.org/archive/2013/RJ-2013-031/.

Goodman-Bacon A (2021). "Difference-in-Differences with Variation in Treatment Timing." *Journal of Econometrics*, **225**(2), 254–277. doi:10.1016/j.jeconom.2021.03.014.

Google (2025). "Agent2Agent Protocol (A2A)." Announced April 2025; transferred to Linux Foundation, URL https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/.

Greene WH (2018). *Econometric Analysis*. 8th edition. Pearson, New York.

Hamilton JD (1994). *Time Series Analysis*. Princeton University Press, Princeton, NJ.

Ho DE, Imai K, King G, Stuart EA (2011). "**MatchIt**: Nonparametric Preprocessing for Parametric Causal Inference." *Journal of Statistical Software*, **42**(8), 1–28. doi:10.18637/jss.v042.i08.

Hollmann N, Müller S, Hutter F (2023). "Large Language Models for Automated Data Science: Introducing CAAFE for Context-Aware Automated Feature Engineering." In *Advances in Neural Information Processing Systems 36 (NeurIPS)*. URL https://arxiv.org/abs/2305.03403.

Hong S, Lin Y, Liu B, Liu B, Wu B, Li D, Chen J, Zhang J, Wang J, Zhang L, Zhang L, Yang M, Zhuge M, Guo T, Zhou T, Tao W, Wang W, Tang X, Lu X, Zheng X, Liang X, Fei Y, Cheng Y, Xu Z, Wu C (2025). "Data Interpreter: An LLM Agent For Data Science." In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 19796–19821. URL https://arxiv.org/abs/2402.18679.

Hothorn T, Zeileis A, Farebrother RW, Cummins C (2024). ***lmtest**: Testing Linear Regression Models*. R package version 0.9-40, URL https://CRAN.R-project.org/package=lmtest.

Hu K (2023). "ChatGPT Sets Record for Fastest-Growing User Base." Reuters. Accessed January 2026, URL https://www.reuters.com/technology/chatgpt-sets-record-fastest-growing-user-base-analyst-note-2023-02-01/.

Hyndman RJ, Khandakar Y (2008). "Automatic Time Series Forecasting: The **forecast** Package for R." *Journal of Statistical Software*, **27**(3), 1–22. doi:10.18637/jss.v027.i03.

IBM Research (2025). "Agent Communication Protocol (ACP)." Launched March 2025; merged with A2A under Linux Foundation, URL https://research.ibm.com/projects/agent-communication-protocol.

Imai K, Ratkovic M (2014). "Covariate Balancing Propensity Score." *Journal of the Royal Statistical Society: Series B*, **76**(1), 243–263. doi:10.1111/rssb.12027.

Kelley J, Dioxus Contributors (2025). "Dioxus: Fullstack Cross-Platform App Framework for Rust." Version 0.7. Accessed March 2026, URL https://dioxuslabs.com.

LeDell E, Poirier S (2020). "H2O AutoML: Scalable Automatic Machine Learning." In *Proceedings of the 7th ICML Workshop on Automated Machine Learning*. URL https://www.automl.org/wp-content/uploads/2020/07/AutoML_2020_paper_61.pdf.

Lee DS, McCrary J, Moreira MJ, Porter J (2022). "Valid t-ratio Inference for IV." *American Economic Review*, **112**(10), 3260–3290. doi:10.1257/aer.20211063.

Li B, Luo Y, Chai C, Li G, Tang N (2024). "The Dawn of Natural Language to SQL: Are We Fully Ready?" *Proceedings of the VLDB Endowment*, **17**(11), 3318–3331. doi:10.14778/3681954.3682003.

Liu W, Huang X, Zeng X, Hao X, Yu S, Li D, Wang S, Gan W, Liu Z, Yu Y, Wang Z, Wang Y, Ning W, Hou Y, Wang B, Wu C, Wang X, Liu Y, Wang Y, Tang D, Tu D, Shang L, Jiang X, Tang R, Lian D, Liu Q, Chen E (2025). "ToolACE: Winning the Points of LLM Function Calling." In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*. URL https://openreview.net/forum?id=8EB8k6DdCU.

Longley JW (1967). "An Appraisal of Least Squares Programs for the Electronic Computer from the Point of View of the User." *Journal of the American Statistical Association*, **62**(319), 819–841. doi:10.1080/01621459.1967.10500896.

Ludwig J, Mullainathan S, Rambachan A (2025). "Large Language Models: An Applied Econometric Framework." *NBER Working Paper*, (33344). doi:10.3386/w33344. URL https://www.nber.org/papers/w33344.

McCullough BD (1998). "Assessing the Reliability of Statistical Software: Part I." *The American Statistician*, **52**(4), 358–366. doi:10.2307/2685442.

McCullough BD, Vinod HD (1999). "The Numerical Reliability of Econometric Software." *Journal of Economic Literature*, **37**(2), 633–665. doi:10.1257/jel.37.2.633.

Ollama Inc (2024). *Ollama: Run Large Language Models Locally*. URL https://ollama.ai.

Pola-rs Contributors (2024). ***polars**: Blazingly Fast DataFrames in Rust and Python*. Version 0.46, URL https://pola.rs/.

Qu C, Dai S, Wei X, Cai H, Wang S, Yin D, Xu J, Wen JR (2025). "Tool Learning with Large Language Models: A Survey." *Frontiers of Computer Science*, **19**(8). doi:10.1007/s11704-024-40678-2.

Renggli C, Ilyas IF, Rekatsinas T (2025). "Fundamental Challenges in Evaluating Text2SQL Solutions and Detecting Their Limitations." *arXiv preprint*. ArXiv:2501.18197, URL https://arxiv.org/abs/2501.18197.

Schick T, Dwivedi-Yu J, Dessì R, Raileanu R, Lomeli M, Zettlemoyer L, Cancedda N, Scialom T (2023). "Toolformer: Language Models Can Teach Themselves to Use Tools." In *Advances in Neural Information Processing Systems (NeurIPS)*. URL https://arxiv.org/abs/2302.04761.

Staiger D, Stock JH (1997). "Instrumental Variables Regression with Weak Instruments." *Econometrica*, **65**(3), 557–586. doi:10.2307/2171753.

Stock JH, Yogo M (2005). "Testing for Weak Instruments in Linear IV Regression." In DWK Andrews, JH Stock (eds.), *Identification and Inference for Econometric Models: Essays in Honor of Thomas Rothenberg*, pp. 80–108. Cambridge University Press, Cambridge. doi:10.1017/CBO9780511614491.006.

Sun L, Abraham S (2021). "Estimating Dynamic Treatment Effects in Event Studies with Heterogeneous Treatment Effects." *Journal of Econometrics*, **225**(2), 175–199. doi:10.1016/j.jeconom.2020.09.006.

Sun M, Han R, Jiang B, Qi H, Sun D, Yuan Y, Huang J (2025). "LAMBDA: A Large Model Based Data Agent." *Journal of the American Statistical Association*. doi:10.1080/01621459.2025.2510000. Online first.

van der Laan MJ, Rose S (2011). *Targeted Learning: Causal Inference for Observational and Experimental Data*. Springer, New York. doi:10.1007/978-1-4419-9782-1.

Wilkinson GN, Rogers CE (1973). "Symbolic Description of Factorial Models for Analysis of Variance." *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, **22**(3), 392–399. doi:10.2307/2346786.

Woods WA (1973). "Progress in Natural Language Understanding: An Application to Lunar Geology." In *Proceedings of the AFIPS National Computer Conference*, pp. 441–450. doi:10.1145/1499586.1499695.

Wooldridge JM (2010). *Econometric Analysis of Cross Section and Panel Data*. 2nd edition. MIT Press, Cambridge, MA.

Wooldridge JM (2025). "Two-Way Fixed Effects, the Two-Way Mundlak Regression, and Difference-in-Differences Estimators." *Empirical Economics*, **69**, 2545–2587. doi:10.1007/s00181-025-02807-z.

Xu Y (2017). "Generalized Synthetic Control Method: Causal Inference with Interactive Fixed Effects Models." *Political Analysis*, **25**(1), 57–76. doi:10.1017/pan.2016.2.

Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan K, Cao Y (2023). "ReAct: Synergizing Reasoning and Acting in Language Models." In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*. URL https://arxiv.org/abs/2210.03629.

Yu T, Zhang R, Yang K, Yasunaga M, Wang D, Li Z, Ma J, Li I, Yao Q, Roman S, Zhang Z, Radev D (2018). "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task." In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3911–3921. doi:10.18653/v1/D18-1425.

Zeileis A (2004). "Econometric Computing with HC and HAC Covariance Matrix Estimators." *Journal of Statistical Software*, **11**(10), 1–17. doi:10.18637/jss.v011.i10.

Affiliation:

Ulrich Matter
Bern University of Applied Sciences, Bern, Switzerland
University of St. Gallen, St. Gallen, Switzerland
E-mail: ulrich.matter@unisg.ch
URL: https://umatter.github.io/

Journal of Statistical Software http://www.jstatsoft.org/
published by the Foundation for Open Access Statistics http://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd